

Hawkeyes: An intelligent honeypot allocation strategy for cyber deception using reinforcement learning

Hien Do Hoang ^{a,b}, Trong-Nghia To ^{a,b}, Ngo Duc Hoang Son ^{a,b}, Khoa Ngo-Khanh ^{a,b},
 Nguyen Tan Cam ^{a,b}, Van-Hau Pham ^{a,b,*}

^a University of Information Technology, Ho Chi Minh City, Vietnam

^b Vietnam National University, Ho Chi Minh City, Vietnam

ARTICLE INFO

Keywords:

Cyber deception
 Honeypot allocation
 Adaptive defense
 Hierarchical reinforcement learning
 Multi-agent systems
 Graph-based modeling

ABSTRACT

Honeypot allocation has emerged as a pivotal strategy in cyber deception. However, existing approaches often face scalability issues, limited coordination, and inadequate consideration of intrusion stages, which constrain their effectiveness in complex attack environments. To address these challenges, this study introduces Hawkeyes, a hierarchical multi-agent reinforcement learning (HMARL) framework for adaptive honeypot allocation. The framework combines a high-level policy, which selects strategies based on intrusion progression and asset criticality, with low-level agents that determine precise placements within grouped regions. Scalability is ensured through action-space reduction via grouping and multi-agent decomposition. Network states are modeled as enriched graphs that capture compromised nodes, topology, and host vulnerabilities, enabling stage-aware and context-rich decision making. Experimental results show that Hawkeyes outperforms baseline RL methods, achieving 15–18% higher trapping efficiency and superior stage-aware defense (48.4% vs. 30.6% efficiency at False Negative Rate of 0.05). It also maintains stable performance under topology and vulnerability variations, while ensuring real-time deployment with sub-5s latency.

1. Introduction

In today's rapidly evolving cybersecurity landscape, attackers continuously develop more sophisticated techniques to infiltrate networks, exfiltrate sensitive data, and disrupt systems. Traditional defenses, such as firewalls and intrusion detection systems, often fall short in the face of advanced persistent threats (APTs), zero-day exploits, and other targeted attacks. As a result, cyber deception has emerged as a proactive defense mechanism that seeks to deceive attackers, lure them into traps, and gather valuable intelligence about their behavior [1,2].

Cyber deception has emerged as a powerful active defense paradigm that complements traditional methods [1–4]. In this approach, defenders intentionally present fake assets (e.g., honeypots) to confuse, delay, and mislead attackers. Honeypots, in particular, can lure adversaries away from real systems and serve as surveillance points. Honeypots incur nearly zero false positives since legitimate users should not interact with them, enabling them to detect even novel (zero-day) attacks that escape signature-based defenses [5]. At the same time, they provide defenders with rich insight into attacker tactics, techniques, and procedures (TTPs) [6]. In this way, cyber deception not only strengthens

resilience but also transforms the defender's role from passive reaction to active engagement and intelligence gathering.

This strategy can be contrasted with Moving Target Defense (MTD), which focuses on reducing the likelihood of successful attacks by dynamically reconfiguring system parameters such as IP addresses, service ports, or network topologies. While MTD increases adversarial uncertainty, it generally does not incorporate honeypot deployment within its reconfiguration process. Consequently, MTD primarily reshapes the attack surface, whereas cyber deception actively engages adversaries by directing them into controlled decoy environments. Integrating honeypot placement into adaptive defense thus represents a relatively under-explored avenue that can combine the complementary strengths of both paradigms, further enhancing attacker confusion and improving overall defensive effectiveness.

Research on honeypot deployment has progressively shifted from static placement toward more adaptive and intelligence-driven allocation strategies. Early studies often adopted fixed or semi-static deployments, sometimes modeled through simplified attacker-defender interaction frameworks. Game-theoretic formulations have been applied to derive optimal honeypot placement policies under resource and

* Corresponding author.

E-mail addresses: hiendh@uit.edu.vn (H. Do Hoang), nghiatt@uit.edu.vn (T.-N. To), sonndh@uit.edu.vn (N.D.H. Son), khoank@uit.edu.vn (K. Ngo-Khanh), camnt@uit.edu.vn (N.T. Cam), haupv@uit.edu.vn (V.-H. Pham).

<https://doi.org/10.1016/j.comnet.2025.111982>

Received 22 August 2025; Received in revised form 21 December 2025; Accepted 28 December 2025

Available online 30 December 2025

1389-1286/© 2025 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

mobility constraints [7,8]. While analytically rigorous, these methods typically rely on oversimplified assumptions regarding network topology and adversary behavior. To address these limitations, researchers have increasingly employed reinforcement learning (RL), enabling state-aware and adaptive honeypot allocation that adjusts to evolving attack dynamics [9–12]. Beyond optimization, recent works emphasize context awareness and operational realism, such as industrial Internet of Things (IIoT) honeypots designed to emulate authentic programmable logic controllers (PLCs) communications or deception-intelligence integration that combines honeypot telemetry with threat intelligence reasoning [13–15]. Comprehensive surveys further underline a clear trend toward adaptive, scalable, and context-aware honeypot systems that extend beyond simple trap deployment to become active components of the broader cyber defense ecosystem [1,16].

Despite these advances, however, existing methods still lack scalability, coordination, and stage-awareness, leaving defenders without practical frameworks for real-world environments [7,8]. Existing game-theoretic approaches often rely on simplified network models, while reinforcement learning methods are limited to single-agent settings and ignore coordination across multiple honeypots. In addition, current strategies rarely consider different stages of intrusion and still face challenges in balancing scalability with realistic modeling. These issues call for a honeypot allocation framework that is adaptive, scalable, stage-aware, and practical for deployment in real-world networks. Moreover, deploying honeypots arbitrarily is not feasible in practice, since each decoy introduces deployment, monitoring, and maintenance overhead. Defenders, therefore, operate under a limited decoy budget, where strategic allocation becomes essential to maximize engagement while minimizing unnecessary costs. This practical constraint underscores the need for adaptive mechanisms that can not only react to attacker behaviors but also optimize resource allocation.

To address these limitations, this study introduces Hawkeyes, a hierarchical reinforcement learning (HRL) framework designed to make honeypot allocation both adaptive and scalable. The framework organizes decision-making into two levels: a high-level policy that selects appropriate allocation strategies depending on the intrusion stage and the criticality of targeted assets, and a low-level policy that determines the exact honeypot location within the chosen region. The proposed architecture enhances scalability by grouping nodes and applying multi-agent decomposition to reduce the action space, while also improving the likelihood of effectively trapping attackers. To further strengthen decision quality, Hawkeyes incorporates a multi-dimensional state representation that captures node status, network topology, and vulnerabilities, enabling more context-aware and robust allocation. Through extensive evaluation, our approach demonstrates strong adaptability and efficiency, positioning it as a practical step toward operationalizing deception-based cyber defense.

In general, the main contributions of this work are as follows:

- Propose a hierarchical reinforcement learning framework that explicitly incorporates intrusion stages into honeypot allocation, enabling defenses to adapt as attacks evolve.
- Improve scalability by leveraging node grouping and multi-agent decomposition to reduce the action space and support coordinated placement in large-scale or resource-limited networks.
- Develop a multi-dimensional state representation that captures node status, network topology, and vulnerabilities, thereby enhancing situational awareness and robustness under diverse threat conditions.
- Conduct extensive evaluation against multiple attack strategies in a simulation environment, demonstrating the framework's adaptability, efficiency, and potential for real-world deployment.

The remainder of this paper is structured as follows. Section 2 provides the necessary background on reinforcement learning techniques. Section 3 formulates the deception deployment problems. Section 4 presents the design of our proposed method. Section 5 presents the experimental setup and evaluation, analyzing performance under diverse

attack scenarios. Section 6 reviews related work on honeypot allocation. Finally, Section 7 concludes the paper and Section 8 outlines potential directions for future research.

2. Background

2.1. Hierarchical reinforcement learning

Hierarchical Reinforcement Learning (HRL) extends conventional reinforcement learning by introducing multilevel policy structures that decompose complex tasks into a hierarchy of subtasks [17,18]. This abstraction facilitates scalable decision-making in environments with large state and action spaces, making HRL particularly well-suited for cyber deception systems such as honeypots, which require adaptive responses to sophisticated attacker behaviors in dynamic environments [19,20].

The core idea of HRL is to organize an agent's policy into two or more layers. A high-level policy selects abstract actions, which define a temporal commitment to a lower-level policy responsible for executing primitive actions in the environment. In the honeypot deployment context, the high-level controller might select an overarching deception objective, such as containment, surveillance, or data exfiltration, while low-level agents carry out concrete tasks like position selection or honeypot deployment. This structure allows for modularity, knowledge reuse, and more efficient learning across different network topologies and attack scenarios [21].

In the context of honeypot-based defense, HRL enables task decomposition and transferability. Sub-policies learned for attacker profiling or decoy interactions can be reused across network segments or attack stages, accelerating the training process and enhancing adaptability. Moreover, by structuring rewards hierarchically, HRL can guide exploration toward long-term defense objectives while avoiding inefficient policy updates [20].

2.2. Multi-Agent reinforcement learning

Multi-Agent Reinforcement Learning (MARL) extends traditional reinforcement learning to environments involving multiple agents that interact with each other and with a shared environment [18,22]. Unlike single-agent settings, each agent in MARL learns a policy to maximize its own cumulative reward in a dynamic and potentially adversarial context [23,24]. MARL has received significant attention due to its applicability in complex, distributed systems, such as swarm robotics, autonomous vehicles, and cybersecurity, where coordination, competition, or both may arise among agents.

Formally, a MARL environment is commonly modeled as a Markov Game (or stochastic game), which generalizes the Markov Decision Process (MDP) to multiple agents. It is defined by a tuple $\langle \mathcal{N}, S, \{\mathcal{A}_i\}_{i \in \mathcal{N}}, \mathcal{P}, \{r_i\}_{i \in \mathcal{N}}, \gamma \rangle$, where:

- $\mathcal{N} = \{1, 2, \dots, n\}$ is the set of agents.
- S is the set of environment states.
- $\mathcal{A}_i$ denotes the action space of agent i .
- $\mathcal{P} : S \times \mathcal{A}_1 \times \dots \times \mathcal{A}_n \rightarrow \Delta(S)$ is the state transition function, mapping joint actions to a distribution over next states.
- $r_i : S \times \mathcal{A}_1 \times \dots \times \mathcal{A}_n \rightarrow \mathbb{R}$ defines the individual reward for agent i .
- $\gamma \in [0, 1)$ is the discount factor.

Each agent i aims to learn a policy $\pi_i(a_i | s)$ that maximizes its expected discounted return:

$$J_i(\pi_i) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_i^t \right] \quad (1)$$

A core challenge in MARL arises from the fact that, during learning, the environment becomes non-stationary from the perspective of any individual agent. This non-stationarity results from the simultaneous policy updates of other agents, making convergence and stability

significantly more difficult to achieve. To address this, several learning paradigms have been proposed, including centralized training with decentralized execution [25,26], parameter sharing, and explicit inter-agent communication for coordination.

In the context of honeypot deployment, MARL offers a powerful framework for coordinating multiple autonomous agents responsible for monitoring and defending different segments of a network. These agents must learn not only optimal local behaviors but also cooperative strategies that enhance the system's overall deception effectiveness. By enabling inter-agent coordination, the network can dynamically respond to attacker behaviors, share threat intelligence efficiently, and make collective decisions that improve robustness and resilience against complex, multi-stage attacks.

3. Modeling the deception deployment problem

3.1. Attacker-defender model

This study introduces an attacker-defender model in which the defender utilizes RL to determine optimal deployment strategies for deception resource allocation dynamically. Unlike traditional methods that rely on static rules or hand-crafted heuristics, our approach enables the defender to autonomously learn how to mislead the attacker, based on observed network security data, without requiring prior knowledge about the attacker's strategy.

The attacker-defender interaction takes place within a structured network environment comprising three key components: the target network $\mathcal{T}$, the attacker $\mathcal{A}$, and the defender $\mathcal{D}$. In this setting, the attacker aims to compromise high-value resources, while the defender strategically places decoys to divert and mislead the attacker along unproductive paths. The RL agent adaptively adjusts the placement of deception resources to maximize the probability of misdirecting the attacker.

3.1.1. Target network structure

The target network $\mathcal{T}$ is formally defined as a triple in Eq. (2), where $\mathcal{N}, \mathcal{E}, \mathcal{G}, \mathcal{D}$ represent the set of nodes, edges, groups of nodes, and deception nodes, respectively.

$$\mathcal{T} = (\mathcal{N}, \mathcal{E}, \mathcal{G}, \mathcal{D}) \quad (2)$$

Node $n_i \in \mathcal{N}$ in the target network represents available resources, such as hosts, servers, or network devices. In addition to specific associated general attributes, such as software versions, vulnerability information, and functional roles, these nodes may contain known security vulnerabilities that can be exploited, which we denote as $\mathcal{V}(n_i)$. Besides, as each node can play various roles in attack strategies, they can be added into two special node subsets of $\mathcal{N}$ called $\mathcal{N}_{init}$ and $\mathcal{N}_{cn}$. The former subset consists of *initial attack nodes* n_{init} that are accessible entry points to external attackers and can be used as the starting position for potential attacks. Meanwhile, the latter stands for *critical nodes* including nodes n_{cn} that have high-value assets available, such as databases or key services, hence are primary targets in most attack scenarios.

Besides actual nodes $\mathcal{N}$, the target network can be equipped with deception nodes (fake nodes) $d_i \in \mathcal{D}$ created to mislead attackers and gather information. The size of $\mathcal{D}$ is limited due to the deployment cost of honeypot resources.

Moreover, nodes can be organized into groups, making up the set $\mathcal{G} = \{g_1, g_2, \dots, g_m\}$, where each group g_i contains nodes that are logically or physically related, such as nodes within the same subnet, trust zone, or functionality domain. Importantly, this grouping is a logical abstraction used only for RL decision-making. It does not modify the underlying network topology or traffic routing. In our framework, deception resources are deployed at the group level rather than individual nodes. This grouping mechanism significantly reduces the action space in reinforcement learning and enables scalable policy learning in large networks.

To denote the connectivity and reachability relationship between nodes, $\mathcal{E}$ is defined as a set of directed edges, in which $e_{ij} = (n_i, n_j)$ exists if there is any communication or logical path from node n_i to n_j . Each edge is associated with two semantic weights of H_{EPSS} and H_{CVSS} regarding the possibility and severity of attacks that can take place on the corresponding connection. While $H_{EPSS}(e_{ij})$ is a weight based on the Exploit Prediction Scoring System (EPSS), indicating the likelihood of exploitation, $H_{CVSS}(e_{ij})$ is a weight derived from the Common Vulnerability Scoring System (CVSS), reflecting the severity of vulnerabilities that can propagate along the edge. These weights are used later to compute attacker movement, estimate risk, and guide deception placement decisions.

Fig. 1 illustrates the structure and attack-defense dynamics within a target network. Initially, the attacker gains access by exploiting vulnerabilities in externally exposed nodes. After the initial compromise, the attacker launches lateral movement attacks through the network in an attempt to reach nodes hosting critical resources. In response, the defender continuously analyzes potential attack paths and strategically deploys deception resources, such as fake nodes or services, within selected groups of nodes. These decoys are intended to mislead the attacker, disrupt their progress, and collect intelligence about their behavior.

It should be noted that, depending on the access policy, there are situations where nodes belonging to one group are allowed to communicate with nodes in another group (e.g., from group g_1 to g_2 , or from g_3 to g_4). In such cases, the connectivity is depicted using group-to-group arrows. Conversely, when the access relationship involves specific nodes, namely, one or more nodes in a group being able to communicate with one or more nodes in another group, the connectivity is illustrated using node-to-node arrows.

3.1.2. Initial compromise and attack flow

In practical network environments, only a subset of nodes, typically remote-access gateways, DMZ services, or externally-facing devices, are exposed to the outside of the network. This subset, denoted as $\mathcal{N}_{init}$, can contain nodes with exploitable vulnerabilities. The attacker begins their intrusion campaign by compromising one of these nodes, referred to as the compromised node $n_{com} \in \mathcal{N}$.

After gaining control of n_{com} , the attacker establishes an internal foothold and attempts to move laterally within the network. This is modeled by traversing the directed edges in $\mathcal{E}$, representing node access relationships. The attacker may first compromise adjacent nodes within the same group and eventually attempt to reach nodes in other groups in $\mathcal{G}$. This flow mimics realistic attack progression, in which the adversary escalates privileges and leverages network connectivity to approach high-value targets located in $\mathcal{N}_{cn}$.

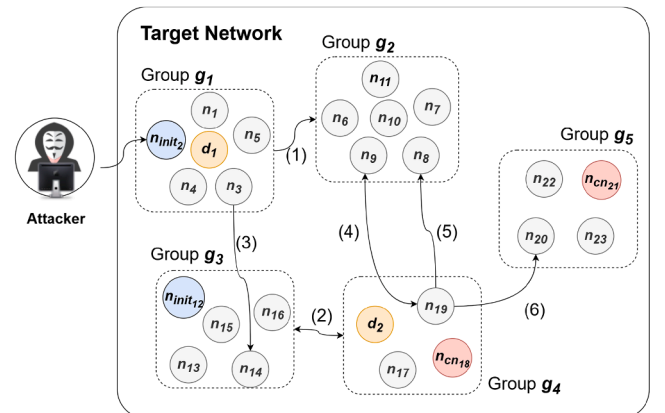


Fig. 1. An illustrative example of the topology of a target network.

3.2. Uncertainty in attack and defense behaviors

In real-world network environments, uncertainty significantly impacts the effectiveness of both attack detection and defense planning. Two major sources of uncertainty must be taken into account.

First, network monitoring systems, including intrusion detection systems (IDS), SIEM, or anomaly-based detectors, are inherently imperfect. These systems may suffer from false negatives (FNR), where actual intrusions go undetected, and false positives (FPR), where benign actions are incorrectly flagged as malicious. If these inaccuracies are ignored, the defender may misjudge the attacker's progress or presence, leading to ineffective responses. Therefore, incorporating this uncertainty into the defense model is essential for designing strategies that are robust under incomplete or noisy observations.

Second, the attacker's strategy is not fixed or observable. Attackers may vary in their level of aggressiveness, stealth, or target preference. For instance, an attacker could focus on lateral movement to explore the network, escalate privileges before striking, or directly attack known high-value nodes. Since defenders cannot precisely predict these strategies, any fixed-rule-based deployment of deception resources becomes ineffective. Accounting for attacker strategy uncertainty enables adaptive defense mechanisms that generalize across a wide range of attack behaviors, improving resilience in adversarial environments.

3.3. Deception resource deployment policy

In our proposed model, the action space of each RL agent is defined as the selection of a group $g \in \mathcal{G}$ in which to deploy a deception node (i.e., honeypot). Once deployed in group g , the deception node becomes logically integrated into the group's topology. Specifically, it is connected to all nodes within the group, and reciprocally, each node in the group is connected to the deception node. Fig. 2 presents the connection between nodes before and after deploying a decoy. This design ensures that the deception node appears as a legitimate participant in intra-group communication and interaction.

This fully-connected deployment within a group is to maximize the deception node's visibility and plausibility. By emulating the connectivity pattern of a typical node in the group, the honeypot becomes indistinguishable from normal resources, increasing the probability that it will be engaged by an attacker. Moreover, full connectivity allows the deception system to monitor more interactions, capture diverse attack behaviors, and respond more adaptively.

A potential concern arising from this design is that if the deception node is compromised, its multiple connections might be exploited as pivot points for lateral movement within the group. While this is a valid observation in conventional network settings, it is significantly mitigated in deception-based defense. Honeypots are not real assets, and they are instrumented, monitored, and often sandboxed. Any interaction with the deception node is considered suspicious and can trigger prevention or response mechanisms. Furthermore, deception platforms such as TrapX DeceptionGrid [27] or Singularity Hologram [28] have adopted similar high-connectivity honeypots precisely to enhance realism and detection effectiveness.

Additionally, the deception node's function is not to relay real traffic but to observe and analyze malicious behavior. It does not hold sensitive data, nor does it grant access to actual resources. It acts as a trap so that any attempt to move laterally through the deception node provides valuable forensic insight and can be used to isolate the attacker.

3.4. Network threat graph

To model potential attack paths within the network, we construct a directed graph $G_{NTG} = (\mathcal{N}, \mathcal{E})$, referred to as the *Network Threat Graph* (NTG). In this graph, nodes in $\mathcal{N}$ represent network assets (e.g., servers, workstations, IoT devices), and each directed edge $e = (n_i, n_j) \in \mathcal{E}$ indicates a potential transition from node n_i to node n_j in an attack sequence.

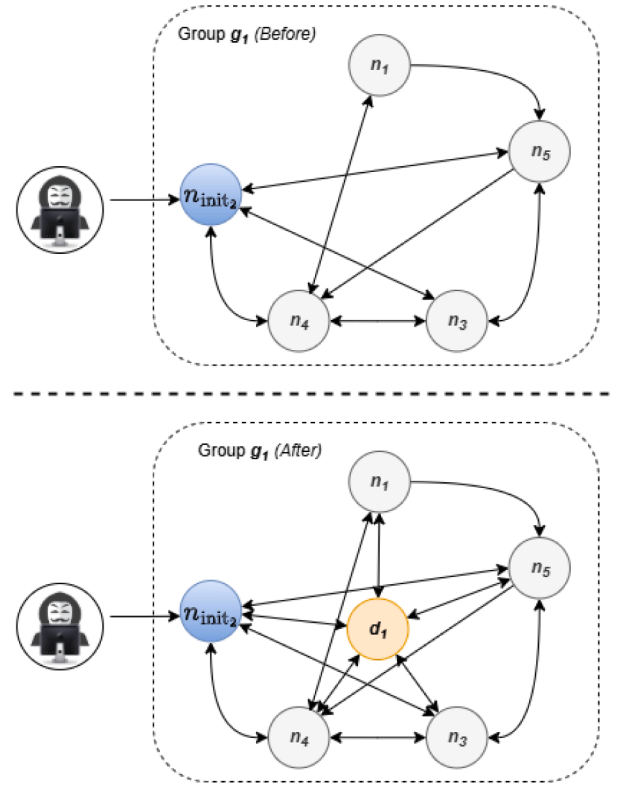


Fig. 2. Illustration of a group before and after deploying a deception node.

The existence of such an edge implies that, under certain conditions, such as the presence of an exploitable vulnerability or lateral movement capability, an attacker can move from n_i to compromise n_j .

Each edge e is associated with a weight tuple:

$$w(e) = (H_{CVSS}, H_{EPSS})$$

where:

- H_{CVSS} denotes the CVSS score, which ranges from 0.0 to 10.0. A higher score indicates a more critical vulnerability.
- H_{EPSS} denotes the EPSS probability. Its value lies within the range $[0, 1]$, with higher values indicating greater real-world exploitation probability.

Fig. 3 depicts a simple example of NTG. Together, H_{CVSS} and H_{EPSS} provide a composite representation of both the technical severity and the practical exploitation risk of each attack transition. This allows NTG to serve as a foundation for evaluating likely attack paths, ranking threats, and guiding defense mechanisms such as honeypot allocation or intrusion response planning.

4. Methodology

In our approach, we focus on solving three problems of deception resource allocation. At first, the RL model must be supplied with a comprehensive set of network data, which is critical for predicting an attacker's current and future movements. With the lack of network information, the model cannot accurately anticipate how attackers will navigate through the network, leading to gaps in defense. We realize that the network data should typically include insights such as the status of compromised nodes, the network's topology, and the vulnerabilities of various hosts. With these inputs, the RL model can intelligently determine the optimal locations for deploying honeypots, increasing the probability of intercepting attackers before they reach valuable resources.

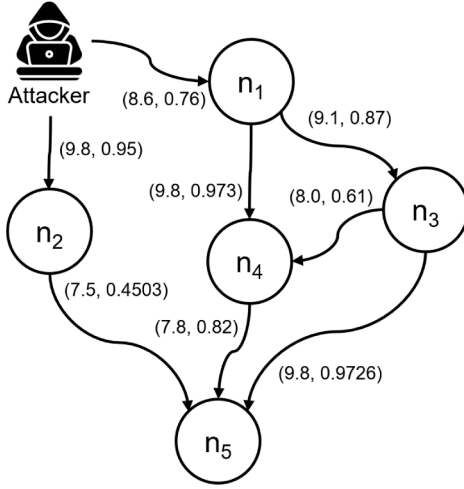


Fig. 3. A simplified network threat graph in which each edge represents a potential attack path, weighted by a pair (H_{CVSS}, H_{EPSS}) .

Secondly, to effectively deceive an attacker, defenders must strategically deploy deception resources within the target network. In the approach discussed by the authors in [9], these resources are deployed after specific nodes. However, when a network contains many nodes and requires many deception resources, this approach significantly increases the action space for the reinforcement learning model. As a result, the RL model faces challenges in identifying the optimal actions, as it must navigate a vast and complex decision space. This not only raises computational costs but also degrades the overall performance of the model, making it harder to achieve efficient and timely honeypot deployment. Our approach involves deploying deception resources within specific groups of hosts. In a network, the number of groups is usually less variable and much smaller than the number of hosts. Therefore, when the number of hosts increases, it has little or no impact on the action space. Additionally, security policies must be established to restrict connections from both nodes and honeypots, ensuring that attackers cannot detect fake resources and limiting the risk of lateral movement attacks.

Finally, the number of deployed honeypots can fluctuate over time. Moreover, while multiple honeypots are deployed, it's possible for a single well-placed honeypot to effectively deceive an attacker, necessitating separate consideration of each honeypot's contribution to the overall defense. Instead of treating honeypots as fully independent learners with separate reward signals, the framework adopts a multi-agent reinforcement learning (MARL) architecture where each agent independently selects a deployment location but jointly optimizes under a shared reward. This shared-reward mechanism ensures that all agents coordinate toward maximizing collective deception effectiveness, rather than competing for individual trapping events. Because the agents share a global reward while maintaining decentralized decision-making, the system supports flexible scaling, adding or removing honeypots without requiring changes to the neural network architecture, while still promoting cooperative placement strategies that increase the likelihood of trapping attackers.

4.1. Overall architecture

The Hawkeyes architecture consists of components depicted in Fig. 4. Specifically, the environment in this framework includes a network monitoring system (NMS) responsible for monitoring network traffic and hosts to identify attacks in the target network. Besides, it also returns states and rewards for the RL agent. The agent is a deep neural network that uses state inputs and decides actions. Then, the Honeypot orchestrator deploys honeypots at corresponding locations based on the given action.

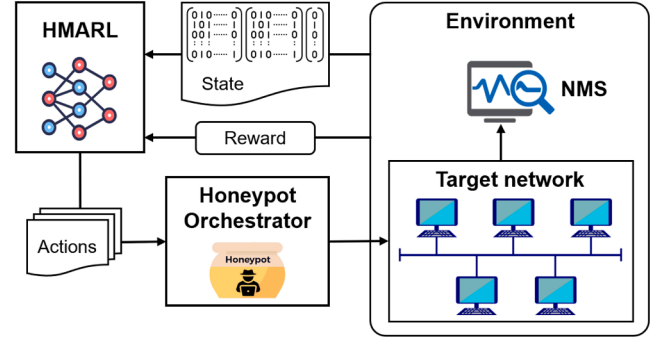


Fig. 4. The overview of Hawkeye's architecture.

4.2. Environment definition

In our approach, the reinforcement learning (RL) agent plays the role of the defender, while the environment consists of both the target network and the attacker. In such an environment, a Network Monitoring System (NMS) is responsible for continuously monitoring the target network to detect suspicious activities and generate real-time alerts. Its alerts help identify compromised nodes, which are directly attacked by an attacker or send out malicious traffic.

Moreover, the environment definition also includes detailed data on the network topology and the vulnerabilities of each host, providing the RL agent with a comprehensive understanding of the target network. This enables the agent to optimize its decision-making when deploying honeypots more effectively. It is important to note that network information may change over time due to factors such as modifications in network policies during maintenance, which can alter connectivities, or the discovery and patching of vulnerabilities. In such cases, the environment should be updated accordingly to ensure the RL agent has the latest network state, allowing it to adapt its defensive strategy and deploy deception resources more effectively.

Besides, we also consider the possible *uncertainty* and seek means to present it in the environment to make it close to the real context. Firstly, since no NMS can perfectly distinguish malicious activities and benign ones without errors, this imperfection of NMS is indicated in the form of two error rates called *FPR* and *FNR*. Secondly, due to the diversity and unpredictability of the possible attack paths of attackers, we assume that in our environment, attackers can decide to attack the next target via one of the five following attacking strategies, denoted $\mathcal{K}$.

- **Highest EPSS-based target selection (HiE):** Based on the EPSS score that reflects the likelihood that a vulnerability can be exploited, this attack strategy focuses on the target whose vulnerabilities are assigned the highest EPSS.
- **Highest CVSS-based target selection (HiC):** In this strategy, attackers are assumed to focus on vulnerabilities with higher severity, presented in CVSS scores. The target host is the one containing the CVE with the highest CVSS among available ones on the network.
- **Random target selection (Ran):** Attackers randomly choose a target in the given network to attack without considering any special properties or criteria.
- **Weighted EPSS-based random target selection (RanE):** Instead of attacking the vulnerability with the highest EPSS, this strategy allows attackers to randomly choose from a list of possible targets with their corresponding weight w_{epss} calculated as in Eq. (3), where n is the number of vulnerabilities on this target. This weight-based randomization ensures that each target can be chosen with a probability of w_{epss} , where a higher EPSS can increase the potential to become a victim.

$$w_{epss} = \frac{\sum_{i=1}^n EPSS_i}{n} \quad (3)$$

- **Weighted CVSS-based random target selection (RanC):** This is mostly the same as the above random strategy using weight, except using CVSS instead of EPSS as in Eq. (4).

$$w_{cvss} = \frac{\sum_{i=1}^n CVSS_i}{n} \quad (4)$$

- **Mixed target selection (Mix):** This 1 aims not to force attackers to a specific attack strategy but allows flexible switching between the above available ones, which makes the prediction and defense more challenging.

4.3. Hierarchical multi-agent reinforcement learning approach

In this research, we adopt a Hierarchical Multi-Agent Reinforcement Learning (HMARL) approach to address the complexity and scalability challenges associated with intelligent honeypot deployment. The hierarchical structure allows us to decompose the decision-making process into two levels: a high-level policy that determines overall deployment strategies, and low-level agents that execute these strategies through specific honeypot placement actions.

The use of hierarchical architecture for reinforcement learning is motivated by its ability to capture long-term objectives and guide agents with high-level strategic decisions. In the context of honeypot deployment, this architecture enables the system to adaptively shift its decoy allocation strategy based on the evolving behavior of attackers and the changing state of the network. Our approach improves decision quality and learning efficiency by abstracting complex decisions into manageable sub-tasks. Simultaneously, we employ a Multi-Agent Reinforcement Learning (MARL) paradigm, where each agent is responsible for controlling the deployment of a single honeypot. These agents operate independently, making localized decisions that are aligned with the overall guidance provided by the high-level policy. This decentralized design enhances scalability and robustness, allowing the system to function effectively in large-scale networks. The hierarchical decision-making architecture for honeypot allocation is illustrated in Fig. 5.

A key advantage of the multi-agent approach is that the action space remains constant regardless of the number of honeypots in the system. In traditional single-agent settings, determining where to place multiple honeypots often requires evaluating all possible combinations of locations. This leads to a combinatorial explosion in the number of actions as the network scales. By contrast, using MARL significantly reduces this complexity, as each agent independently decides where to deploy its assigned honeypot based on a shared high-level strategy. As a result, adding or removing honeypots does not increase the size of the action space. Each new honeypot is managed by a newly instantiated agent that inherits the learned policy of existing agents, eliminating the need to retrain the entire model. This leads to lower computational overhead and enables rapid adaptation in dynamic network environments.

4.3.1. High-level strategy

In our proposed framework, the high-level policy acts as a meta-controller that is responsible for determining the strategic direction for

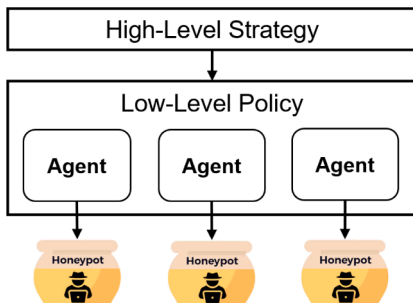


Fig. 5. HMARL architecture for honeypot allocation.

honeypot deployment based on the current state of the network. This layer employs a rule-based gating mechanism designed to ensure stable coordination among low-level agents. Formally, we define the high-level policy as a mapping function $\pi_{high} : S \rightarrow \mathcal{O}$, where S is the set of possible system states, and $\mathcal{O}$ represents the set of strategic goals, each corresponding to a specific area or group of nodes where honeypots should be deployed. We define three policies at this level below.

- **Initial-attack node allocation strategy $\mathcal{O}_{init}$:** This policy prioritizes deploying honeypots at initial attack nodes that are more likely to be exposed to external attacks.
- **Critical node allocation strategy $\mathcal{O}_{crit}$:** This strategy shifts to deploying honeypots within node groups that contain or are adjacent to these high-value resources if an attacker has successfully breached the network and is able to access critical assets.
- **Strategy of delegation to low-level policy $\mathcal{O}_{low}$:** In all other scenarios, the system delegates the decision-making to the low-level policy (π_{low}), which selects the optimal deployment location based on prior training experience and environmental feedback.

Based on the current system state $s \in S$, the high-level policy π_{high} selects one of the strategies that are summarized in Eq. (5).

$$\pi_{high} = \begin{cases} \mathcal{O}_{init}, & \text{if } Compromised(s) = \emptyset; \\ \mathcal{O}_{crit}, & \text{if } ReachableFromAttacker(s) = True; \\ \mathcal{O}_{low}, & \text{otherwise.} \end{cases} \quad (5)$$

Where:

- $Compromised(s)$ returns the set of nodes identified as compromised in state s .
- $ReachableFromAttacker(s)$ evaluates whether any critical node $n_{cn} \in \mathcal{N}_{cn}$ is reachable from a compromised node in the current state s , based on network topology and current connectivity.

Fig. 6 illustrates the internal flow of the gating mechanism, providing a clearer understanding of Eq. (5).

4.3.2. State

To ensure that the reinforcement learning (RL) model can effectively predict optimal locations for the deployment of honeypots, it must be equipped with comprehensive information on compromised nodes, the network topology, and the vulnerabilities of individual hosts. Besides, to be usable in the RL model, those pieces of information are presented in the form of numeric matrices with various dimensions based on their purposes.

$$S_t = (\mathcal{M}_{com}, \mathcal{M}_{con}, \mathcal{M}_{vul}) \quad (6)$$

First, compromised nodes serve as an essential input, revealing which parts of the network have already been compromised by an attacker. Using these compromised nodes, the RL model can figure out potential attack paths, predicting the next likely steps attackers will take

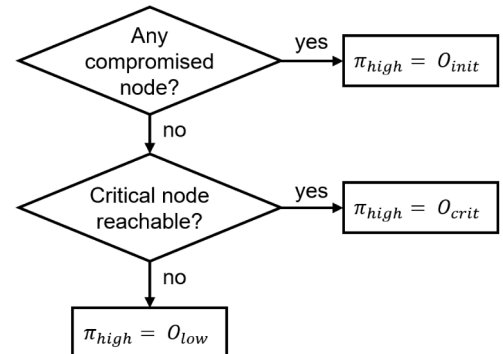


Fig. 6. Illustration of the gating mechanism described in Eq. 5.

as they move laterally through the network. In the scope of our framework, a host is considered compromised when it is attacked successfully or sends out malicious traffic, both of which can lead to a security alert returned from NMS. With all k nodes on the network at the period t , a 1-dimensional *Compromising matrix* M_{com} as in Eq. (7) is produced, in which ω_i^t is assigned to 1 and 0 indicating the status of being attacked successfully or not of host i , respectively.

$$M_{com_t} = (\omega_1^t, \omega_2^t, \dots, \omega_k^t) \quad (7)$$

Moreover, a network threat graph for the network topology is also utilized to form a matrix representing the available connections between hosts and their potential to be used in attack paths. Due to the bidirectional nature of connections, this is a 2-dimension *Connection matrix* M_{con} (Eq. (8)), where $c_{i,j}^t$ takes the value of 0 or 1 to illustrate the available link from node i to node j and $c_{j,i}^t$ demonstrates the opposite direction of the connection.

$$M_{con_t} = \begin{pmatrix} c_{1,1}^t & c_{1,2}^t & \dots & c_{1,k}^t \\ c_{2,1}^t & c_{2,2}^t & \dots & c_{2,k}^t \\ \vdots & \vdots & \ddots & \vdots \\ c_{k,1}^t & c_{k,2}^t & \dots & c_{k,k}^t \end{pmatrix} \quad (8)$$

Besides, the network threat graph is also combined with the CVSS and EPSS regarding possible vulnerabilities in the host to result in a matrix M_{vul} providing the potentiality of attacking a host from another one using a given connection. This matrix has 3 dimensions, where the first one plays the role of identifying connections between hosts, and the other two dimensions include a representative array $[H_{CVSS}, H_{EPSS}]$ for destination hosts. Given a host with n CVEs, in which k vulnerabilities can obtain the highest privilege such as *root* (Linux/Unix) or *system* (Windows), the corresponding H_{CVSS} and H_{EPSS} values of connection destined to it are calculated as in Eq. (9) and Eq. (10). Note that α is a customizable weight assigned to vulnerabilities that gain lower privileges. Since attackers seem to intend to obtain the highest privilege via *root* or *system*, α is designed to lie in the range of [0,1] to indicate lower priority, but a higher α can be used when focusing more on low-privilege users.

$$H_{CVSS} = \alpha \sum_{i=1}^{n-k} CVSS_i + \sum_{j=1}^k CVSS_j \quad (9)$$

$$H_{EPSS} = \alpha \sum_{i=1}^{n-k} EPSS_i + \sum_{j=1}^k EPSS_j \quad (10)$$

Two terminal states are used to represent the outcome of the defense: S_{succ} indicates a successful defense, while S_{fail} indicates a failure. If the deployment policy successfully traps the attacker before they reach critical assets, the environment transitions to the S_{succ} state. In another case, if the attacker manages to penetrate the defender's high-value resources is considered to have failed, and the state changes to S_{fail} .

4.3.3. Action

In our attacker-defender model, each RL agent is associated with a deception node $d \in D$. The role of the agent is to determine an appropriate group $g \in G$ in which to deploy the honeypot. Therefore, the action space $\mathcal{A}$ is defined as the set of available groups:

$$\mathcal{A} := G$$

An action $a_d \in \mathcal{A}$ represents the decision to deploy deception node d into group g . This formulation allows each agent to learn an allocation policy tailored to the network context and attack progression.

4.3.4. Reward

At each time step t , an agent controlling deception node $d \in D$ observes the current environment state S_t and selects an action $a_{d_t} \in \mathcal{A}$, which corresponds to deploying the honeypot into a selected group $g \in G$. As a result of this action, the environment transitions to a new

state S_{t+1} , and a reward r_t is computed based on the effectiveness of the deployment.

The reward function is defined as:

$$r_t = \begin{cases} 1, & \text{if } S_{t+1} = S_{succ}, \\ -1, & \text{if } S_{t+1} = S_{fail}, \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

To foster coordinated behavior among agents, a shared reward is used across all deception agents. That is, all agents receive the same reward r_t at time t , regardless of which agent performed the action. This collective reward structure encourages agents to learn cooperative strategies, where the deployment of each honeypot contributes to the overall defense effectiveness, rather than optimizing for isolated, potentially conflicting, or duplicating objectives. Sharing the reward aligns agent incentives toward a global goal, maximizing deception success and minimizing critical asset compromise across the entire network [29,30].

4.4. Workflow of the hawkeyes framework

Algorithm 1 presents the training workflow of Hawkeyes, in which the system consists of a high-level policy (π_{high}) and multiple low-level policies (π_{low_d}) corresponding to each agent $d \in D$. At each training iteration, if an episode terminates, the environment ENV is reset using the target network $\mathcal{T}$. At every step, the system observes the current state S_t , and π_{high} selects a high-level strategy:

- If the selected strategy is $\mathcal{O}_{init}$, each agent is restricted to choose a group of nodes intersecting with the set $\mathcal{N}_{init}$.
- If the selected strategy is $\mathcal{O}_{crit}$, each agent is restricted to choose a group of nodes intersecting with the set $\mathcal{N}_{crit}$.
- If the selected strategy is $\mathcal{O}_{low}$, agents are free to select any group of nodes from the entire G .

Each agent d employs its policy π_{low_d} to choose an action a_d satisfying the imposed constraints (if any), and all selected actions are aggregated into $\mathcal{A}$. The system then deploys deception nodes according to $\mathcal{A}$, the attacker applies a strategy from $\mathcal{K}$, and the NMS monitors the network with false positive and false negative rates (FPR, FNR) to generate an alert ϕ . The environment transitions to a new state S_{t+1} , and a reward r_t is assigned, followed by Eq. (11).

During training, each agent continuously collects interaction outcomes, including states, actions, and rewards. At the end of an episode, the agent consolidates its own experiences and invokes the `UpdatePolicy()` function to refine its low-level policy π_{low_d} . This process enables the policy to adjust based on both the recent outcomes and the accumulated history of interactions. The adaptation loop is repeated until the training horizon T_{max} is reached. At that point, the framework returns the optimized set of learned low-level policies $\{\pi_{low_d}\}_{d \in D}$.

To provide an intuitive overview of the system's operational workflow, Fig. 7 depicts a sequence loop. This timeline representation emphasizes the sequential interaction among components and highlights how the model maintains a continuous cycle during both training and execution.

4.5. Honeypot orchestrator

The Honeypot Orchestrator acts as the execution layer of the proposed framework, which is responsible for translating placement decisions into concrete network configurations. Once the high-level and low-level agents determine the optimal deployment locations, the orchestrator coordinates the actual instantiation of honeypots across the target network. This process involves two primary tasks: (i) configuring underlying network infrastructure components, such as routers, switches, and firewalls, to route and isolate traffic toward designated deception zones, and (ii) deploying and initializing honeypot instances with appropriate service profiles and interaction levels. By ensuring that honeypots are placed within the assigned groups, the orchestrator enforces

Algorithm 1 Training workflow of Hawkeyes.

```

1: Input: target network  $\mathcal{T}$ , max steps  $T_{\max}$ , attacker strategy  $\mathcal{K}$ , NMS
   error rates (FPR, FNR), deception nodes  $\mathcal{D}$ 
2: Output: low-level agent policies  $\{\pi_{low_d}\}_{d \in \mathcal{D}}$ 
3:  $t \leftarrow 0$ 
4:  $done \leftarrow True$   $\triangleright$  Return True if an episode finished.
5: while  $t < T_{\max}$  do
6:    $\mathcal{A}_t \leftarrow \emptyset$ 
7:   if  $done = True$  then  $\triangleright$  Initialize a new episode
8:     Initialize environment  $ENV$  from  $\mathcal{T}$ 
9:      $done \leftarrow False$ 
10:  end if
11:   $S_t \leftarrow GetStateFromEnvironment(ENV)$ 
12:   $\pi_h \leftarrow \pi_{high}(S_t)$   $\triangleright$  Select high-level strategy
13:  if  $\pi_h = \mathcal{O}_{init}$  then
14:    for each agent  $d \in \mathcal{D}$  do
15:       $a_{d_t} \leftarrow \pi_{low_d}(S_t \mid \{g \in \mathcal{G} \mid g \cap \mathcal{N}_{init} \neq \emptyset\})$ 
16:       $\mathcal{A}_t$  appends  $a_{d_t}$ 
17:    end for
18:  else if  $\pi_h = \mathcal{O}_{crit}$  then
19:    for each agent  $d \in \mathcal{D}$  do
20:       $a_{d_t} \leftarrow \pi_{low_d}(S_t \mid \{g \in \mathcal{G} \mid g \cap \mathcal{N}_{crit} \neq \emptyset\})$ 
21:       $\mathcal{A}_t$  appends  $a_{d_t}$ 
22:    end for
23:  else  $\triangleright \pi_h = \mathcal{O}_{low}$ 
24:    for each agent  $d \in \mathcal{D}$  do
25:       $a_{d_t} \leftarrow \pi_{low_d}(S_t)$   $\triangleright$  Agent selects group  $g \in \mathcal{G}$ 
26:       $\mathcal{A}_t$  appends  $a_{d_t}$ 
27:    end for
28:  end if
29:  Deploy deception nodes by  $\mathcal{A}_t$ 
30:  NMS observes  $ENV$  with FPR & FNR and returns an observa-
   tion/alert  $\phi$ 
31:   $ENV$  transitions to the next state according to  $(\mathcal{K}, \phi)$ 
32:   $S_{t+1} \leftarrow GetStateFromEnvironment(ENV)$ 
33:   $r_t \leftarrow \begin{cases} 1 & S_{t+1} = S_{succ} \\ -1 & S_{t+1} = S_{fail} \\ 0 & \text{otherwise} \end{cases}$ 
34:   $done \leftarrow (S_{t+1} \in \{S_{succ}, S_{fail}\})$   $\triangleright$  True if matched
35:  if  $done = True$  then  $\triangleright$  The episode finished
36:    for each agent  $d \in \mathcal{D}$  do
37:       $\pi_{low_d} \leftarrow UpdatePolicy(d)$ 
38:    end for
39:     $\mathcal{R} \leftarrow \emptyset$ 
40:  end if
41:   $t \leftarrow t + 1$ 
42: end while
43: return  $\{\pi_{low_d}\}_{d \in \mathcal{D}}$ 

```

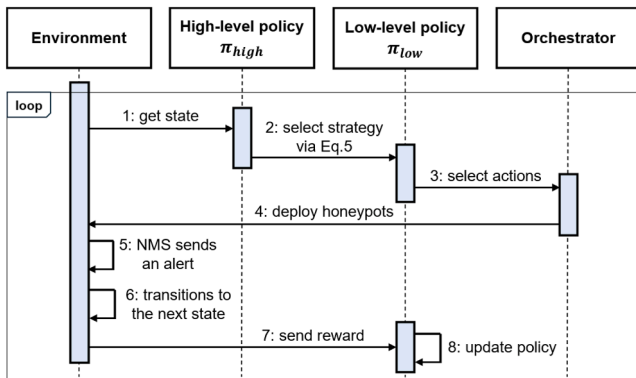


Fig. 7. Timeline of component interactions during training and execution.

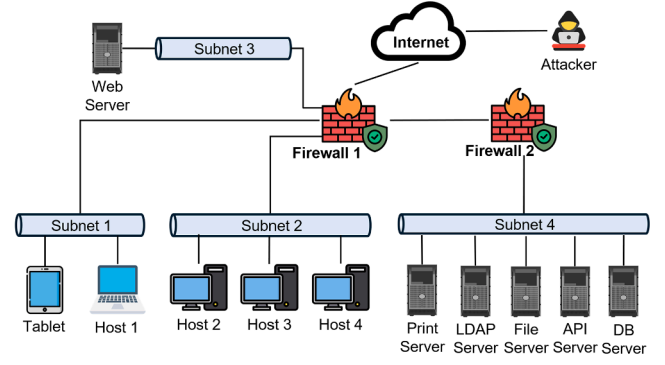


Fig. 8. Network topology for experiments.

both logical and physical consistency between strategic decisions and operational environments.

To mitigate operational risks, all deployed deception nodes operate within isolated sandbox environments under continuous monitoring. Outbound traffic from these honeypots is strictly inspected to prevent potential lateral movement or misuse as pivot points. Furthermore, the orchestrator dynamically reconfigures network devices to restrict or block suspicious traffic once anomalous behaviors are detected, thereby ensuring both containment and operational safety throughout the deception infrastructure.

5. Experiments and evaluation

5.1. Experimental settings

The experiments were conducted on a workstation running Ubuntu 22.04, an Intel Core i7-7700 CPU, 32 GB of RAM, Python 3.10, and PyTorch 2.1. The Advantage Actor-Critic (A2C) algorithm was employed for agent training. A2C was chosen because its synchronous update scheme is well-suited for coordinating multiple agents, while its policy-value function structure enables stable learning in dynamic environments with frequent state transitions, as in the Hawkeyes workflow. The A2C network architecture includes four fully connected layers with decreasing dimensions (2048-1024-512-64), each followed by ReLU activations. The actor head outputs an action probability distribution via a softmax layer, while the critic head estimates the state value through a linear layer. The details of training hyperparameters are shown in Table 1.

Note that, in our experiments, training progress is measured by the number of attacker steps rather than by episode counts. Each episode may require a different number of steps for the attacker to reach its objective. Using step-based measurement provides a more consistent and fine-grained view of the learning process, allowing convergence and performance trends to be evaluated independently of episode duration.

Fig. 8 illustrates the network topology used in the experimental setup. The topology is organized into four subnet layers, each representing a distinct group of nodes in which honeypots can be deployed. Among these, Subnet 4 contains a database server designated as a critical asset requiring enhanced protection. We employed two widely used vulnerability scanning tools: OpenVAS and Nuclei to actively probe the network, perform vulnerability enumeration, and collect detailed information about each host. The list of identified vulnerabilities and information for all hosts and two honeypots in the experimental environment is presented in Table 2 (Data collected on February 2025).

5.2. Evaluation criteria

We define Defense Success Rate (DSR) as the proportion of successful defense outcomes over the total number of attack trials when the defender deploys deception nodes according to the policy π . Let N_{succ}

Table 1
Summary of training hyperparameters for the A2C agent.

Parameter	Value	Description
Learning rate	1e-3	Step size for gradient updates
Discount factor (γ)	0.99	Future reward discount
Optimizer	Adam	Optimization algorithm
Value loss coefficient (c_v)	0.5	Weight for critic loss
Training steps	30,000	Number of attacker steps for training process

Table 2
Information and vulnerabilities of hosts and honeypots.

No.	Hostname	Description	CVE ID	Attack Type	Required privilege of source host	Obtained privilege of target host	CVSS Score	EPSS Score
1	Tablet	iOS 12	CVE-2019-8689	Remote	User	Root	8.8	0.9392
		Safari	CVE-2018-4441	Remote	User	Root	8.8	0.9183
2	Host 1	Google Chrome	CVE-2019-13720	Remote	User	User	8.8	0.9703
		Windows 8.1	CVE-2019-1458	Local	User	System	7.8	0.9714
3	Host 2	Windows 10	CVE-2018-8174	Remote	User	User	7.5	0.9666
			CVE-2018-8550	Local	User	System	7.8	0.9556
			CVE-2018-8626	Remote	User	System	9.3	0.088
4	Host 3	Windows 10	CVE-2018-8174	Remote	User	System	7.5	0.9666
			CVE-2018-8550	Local	User	System	7.8	0.9556
			CVE-2018-8626	Remote	User	System	9.3	0.088
5	Host 4	Windows 10	CVE-2018-8174	Remote	User	System	7.5	0.9666
			CVE-2018-8550	Local	User	System	7.8	0.9556
			CVE-2018-8626	Remote	User	System	9.3	0.088
6	Print Server	Windows Server 2019	CVE-2021-34527	Remote	User	System	8.8	0.9584
7	LDAP Server	Windows Server 2019	CVE-2021-40444	Remote	User	System	8.8	0.9693
			CVE-2021-34498	Local	User	System	7.8	0.8764
8	File Server	Samba on Ubuntu 20	CVE-2021-44142	Remote	User	Root	9	0.1805
		Snapd	CVE-2021-44731	Local	User	Root	7.8	0.1292
9	Web Server	PHP on Linux	CVE-2019-11043	Remote	User	Root	9.8	0.9669
		Drupal	CVE-2019-6340	Remote	User	User	8.1	0.9737
10	DB Server	phpMyAdmin	CVE-2018-12613	Remote	User	User	8.8	0.9732
		Ubuntu 18 LTS	CVE-2021-3156	Local	User	Root	7.8	0.9637
11	Honeypot 1	Windows Server 2019	CVE-2019-0887	Remote	User	System	8.8	0.9732
		PHP	CVE-2019-11043	Remote	User	System	9.8	0.9669
12	Honeypot 2	Windows Server 2019	CVE-2019-0887	Remote	User	System	8.8	0.9732
		phpMyAdmin	CVE-2018-12613	Remote	User	User	8.8	0.9732

denote the number of trials in which the defender successfully traps the attacker, and let N_{total} denote the total number of attack attempts. The DSR is formally expressed as Eq. (12).

$$\text{DSR} = \frac{N_{\text{succ}}}{N_{\text{total}}} \quad (12)$$

A higher DSR indicates that the resource deployment policy is effective in luring attackers into deception traps.

5.3. Training efficiency

To evaluate the computational efficiency of the proposed approach with the A2C-based model for network topology in Fig. 8, we measured the total wall-clock training time across different numbers of attacker steps. As shown in Fig. 9, the training time increases almost linearly with the number of steps, indicating stability of the learning process. Specifically, the model required approximately 4.99 seconds for 500 steps, 44.77 seconds for 5000 steps, and 259.87 seconds for 30,000 steps. The near-linear growth also suggests that the synchronous update mechanism of A2C efficiently utilizes available hardware resources, ensuring practical feasibility for large-scale training.

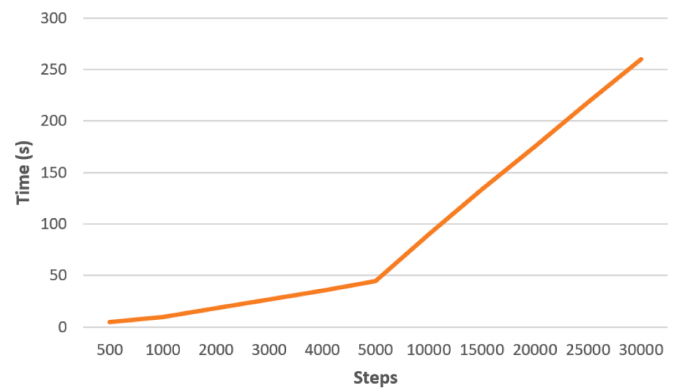


Fig. 9. Training time of the A2C-based model for different numbers of attacker steps.

5.4. Experimental scenarios

To rigorously evaluate the performance of the proposed approach, four experimental scenarios are constructed, each designed to capture

a distinct operational aspect of the system. These scenarios are formulated to emulate representative conditions that frequently arise in practical network environments. The objective is to provide a comprehensive evaluation of the method's adaptability, practicality, and robustness across varying circumstances.

It is important to note that the first three scenarios share identical FNR and FPR parameters. Specifically, the evaluated (FNR, FPR) pairs are: (0.05, 0.02), (0.10, 0.06), (0.15, 0.10), and (0.20, 0.14).

5.4.1. Scenario 1: Evaluating defense strategies under diverse adversarial behaviors

In the first experimental scenario, the objective is to assess the performance of the proposed method under diverse adversarial behaviors. The attacker is configured to adopt six distinct attack policies: HiE, HiC, Ran, RanE, RanC, and Mix (mentioned in Section 4.2). For benchmarking purposes, two baseline strategies are considered: (i) *Random Policy* (Random), in which honeypots are deployed arbitrarily across the network, and (ii) *Critical Policy* (Critical), in which deception nodes are positioned to protect critical nodes only. The experiments are conducted on the network topology illustrated in Fig. 8, where access policies are defined as follows: the Web Server is publicly accessible from the Internet, and hosts in Subnet 1 are allowed to reach the Internet, making them potential entry points for adversarial activities. Hosts in Subnet 1 can also access the Web Server and, due to a misconfiguration, are further able to connect to hosts in Subnet 2. In addition, hosts in Subnet 2 are permitted to access servers located in Subnet 3 and Subnet 4, while the Web Server is allowed to connect to the Database Server. For each configuration, results are averaged over ten independent trials, with each trial consisting of $N_{\text{total}} = 1,000$ attack attempts.

5.4.2. Scenario 2: Validating the necessity of high-level strategy

In this scenario, we examine the role of high-level policy in guiding honeypot placement decisions. Specifically, we compare system performance with and without the use of such policies. This experiment highlights the strategic advantage provided by high-level coordination in improving monitoring efficiency and responsiveness to threats.

Both the proposed hierarchical multi-agent RL (HMARL) and the baseline non-hierarchical multi-agent RL (MARL) share the same actor-critic backbone (A2C) and are trained under identical settings as in Scenario 1.

The key distinction is that HMARL implements a high-level meta-controller to coordinate low-level agents based on intrusion stages, while MA2C agents act independently without inter-agent coordination. This comparison isolates the effect of the hierarchical mechanism, allowing us to assess whether the performance improvements originate from hierarchical structuring rather than differences in the underlying RL algorithm.

5.4.3. Scenario 3: Assessing adaptability under network changes

In the third scenario, we assess the adaptability of the proposed model when the network environment changes, without retraining the RL component. The experimental setup follows the same configuration as Scenario 1, but with modified access rules and vulnerabilities to reflect a realistic, evolving environment. Specifically, hosts in Subnet 2 are now allowed to access the Internet, while hosts in Subnet 1 can no longer connect through Subnet 2 due to updated administrative policies. In addition, the hosts in Subnet 2 have been patched against their previous vulnerabilities but remain exposed to two new ones: CVE-2021-34527 (CVSS: 8.8, EPSS: 0.9584) and CVE-2021-34498 (CVSS: 7.8, EPSS: 0.8764).

The objective of this scenario is to examine whether the model can maintain defensive effectiveness under such network and vulnerability changes, thereby demonstrating its resilience and adaptability. In addition, we compare our method with an approach that only considers

node compromise states as input, following the study in [9], denoted as SO. Results are reported for both the previous network topology (Old) and the modified one (New), to highlight the advantage of our proposed method under changing environments.

5.4.4. Scenario 4: Large-scale evaluation of grouping and reward-sharing mechanisms

This scenario evaluates the effectiveness and scalability of the proposed approach in a larger and more complex network environment, and examines the impact of grouping granularity and reward-sharing mechanisms on defense performance. The simulated network consists of 100 hosts distributed across 8 subnets, with 3 designated critical nodes representing high-value assets. A total of 5 honeypots are available for deployment, reflecting realistic resource constraints in operational networks.

We compare the proposed method (*Ours*) against three variants to isolate the effects of grouping and inter-agent reward sharing. In *Ours*, honeypot allocation is performed using the HMARL framework, where each subnet is treated as a single group. Low-level agents within each group share a common reward signal, encouraging cooperative behavior and coordinated honeypot placement across the network.

The first variant, denoted as *G16*, follows the same HMARL architecture and reward-sharing scheme as *Ours*, but adopts a finer-grained grouping strategy by dividing each subnet into two groups, resulting in a total of 16 groups. This configuration is designed to assess how increased grouping granularity affects learning stability, coordination overhead, and overall defense effectiveness.

The second variant, referred to as *NoG*, employs HMARL with shared rewards but removes the concept of grouping entirely. In this setting, honeypots are placed behind individual nodes, and only the directly connected node is aware of the deployed honeypots. This comparison aims to clarify the role of group-based honeypot placement in enhancing deception visibility and collaborative defense, relative to isolated and node-centric deployment strategies.

The third variant, labeled *NoS*, retains the group-based honeypot placement of *Ours* but disables reward sharing among agents. In this configuration, only the agent successfully trapping an attacker receives a reward, while other agents remain unrewarded. This setting is introduced to evaluate the importance of shared rewards in fostering cooperative behavior and preventing myopic or selfish policies among decentralized agents.

5.4.5. Scenario 5: Demonstrating practical deployability of the proposed approach

In the fifth scenario, we aim to demonstrate the practical deployability of our proposed defense framework. Unlike the previous scenarios that primarily focus on effectiveness and adaptability, this experiment emphasizes the feasibility of end-to-end deployment in a real-world setting. The evaluation is based on 10 independent trials, with measurements taken at specific time milestones. All time values are measured from the start of an attack ($t = 0$). Specifically, *Attack time* marks the moment the adversary initiates the attack, *Detection time* denotes when an attack is detected by NMS, *Decision time* represents the interval required for the RL agents to compute the optimal honeypot locations, and *Deployment time* indicates when deception nodes are completely deployed in the target network.

In our setup, honeypots with pre-installed vulnerabilities are hosted on virtual machines (VMs), and deployment involves the logical reconfiguration of these VMs to the newly assigned network positions rather than instantiating new VMs or containers. This approach allows for rapid deployment while preserving the integrity of the pre-configured honeypot environment.

5.5. Experimental results

5.5.1. Theoretical analysis for action-space reduction via grouping and multi-agent decomposition

Let $N = |\mathcal{N}|$ denote the number of nodes, $G = |\mathcal{G}|$ the number of groups after clustering, and $D = |\mathcal{D}|$ the number of deployed deception nodes. If placement is defined at node granularity, the per-agent action space is of size N ; after grouping, it reduces to G (with $G \leq N$).

In the single-agent setting, allocating D honeypots requires exploring a combinatorial or exponential action space. If the reuse of nodes is allowed, the joint action space is of size

$$N^D,$$

while if placements must be distinct, the size becomes

$$\binom{N}{D} = \frac{N!}{D!(N-D)!}.$$

Both cases quickly become intractable as N and D grow.

In contrast, under our proposed grouping and multi-agent decomposition, each agent is responsible for deploying exactly one honeypot. Consequently, the action space per agent is only G , independent of D . Since $G \ll N^D$ and typically also $G \ll \binom{N}{D}$, this transformation drastically reduces the search space from combinatorial/exponential scale down to linear in the number of agents. For example, with $N = 100$ nodes and $D = 5$ honeypots, the single-agent unordered case requires $\binom{100}{5} \approx 7.5 \times 10^7$ possibilities, whereas in the multi-agent setting with $G = 10$ groups, each agent only needs to select among 10 options.

5.5.2. Scenario 1

The results of Scenario 1, shown in Fig. 10, provide a comprehensive evaluation of the defense strategies under diverse adversarial behaviors. Across all configurations, our proposed method consistently outperforms the baseline strategies, particularly against randomized attacks (Ran, RanE, RanC, and Mix), where it achieves detection and mitigation rates up to 10–15% higher than both the *Random* and *Critical*

policies. For example, under the Ran case with $\text{FNR} = 0.1$, our approach achieves over 85% success compared to less than 75% for the baselines. While the *Critical* strategy demonstrates reasonable effectiveness in protecting highly valuable assets (e.g., yielding nearly 100% defense against the HiC policy), it remains less adaptive to adversaries that deviate from critical-node targeting, resulting in noticeably lower overall performance. By contrast, our method maintains robustness across all attack types, preserving a significant margin of advantage across different detection settings.

The comparison between the results of HiE and HiC highlights an important observation. The relatively low success rate against HiE stems from the fact that this policy guides the attacker toward nodes with high EPSS scores. Since the CVEs deployed on the honeypots tend to have lower EPSS values than most nodes in the network, the probability of the attacker targeting the honeypot under HiE is inherently low. In contrast, under the HiC policy, attackers are directed toward nodes with high CVSS scores. Because the vulnerabilities placed on honeypots have higher CVSS scores than the majority of other nodes, the likelihood of attackers engaging with the honeypot is substantially increased, which explains the near-perfect trapping success in this case. These findings suggest that, for attack policies heavily influenced by vulnerability scoring (EPSS or CVSS), the trapping effectiveness depends more on the inherent properties of the vulnerabilities on the honeypots than on the allocation strategy itself.

Moreover, the results indicate that variations in FNR and FPR do not significantly degrade the relative advantage of our approach. Even when FNR increases from 0.05 to 0.2, the proposed method sustains consistently higher resilience compared to the baselines. As an additional observation, Table 3 shows that the DSP values across 10 runs, in the case of $\text{FNR} = 0.05$ and $\text{FPR} = 0.02$, remain consistently stable for all attack types, with minimal fluctuations. This demonstrates that the integration of high-level strategies enables the system to anticipate better and adapt to heterogeneous attack behaviors, thereby offering more reliable protection in practical settings.

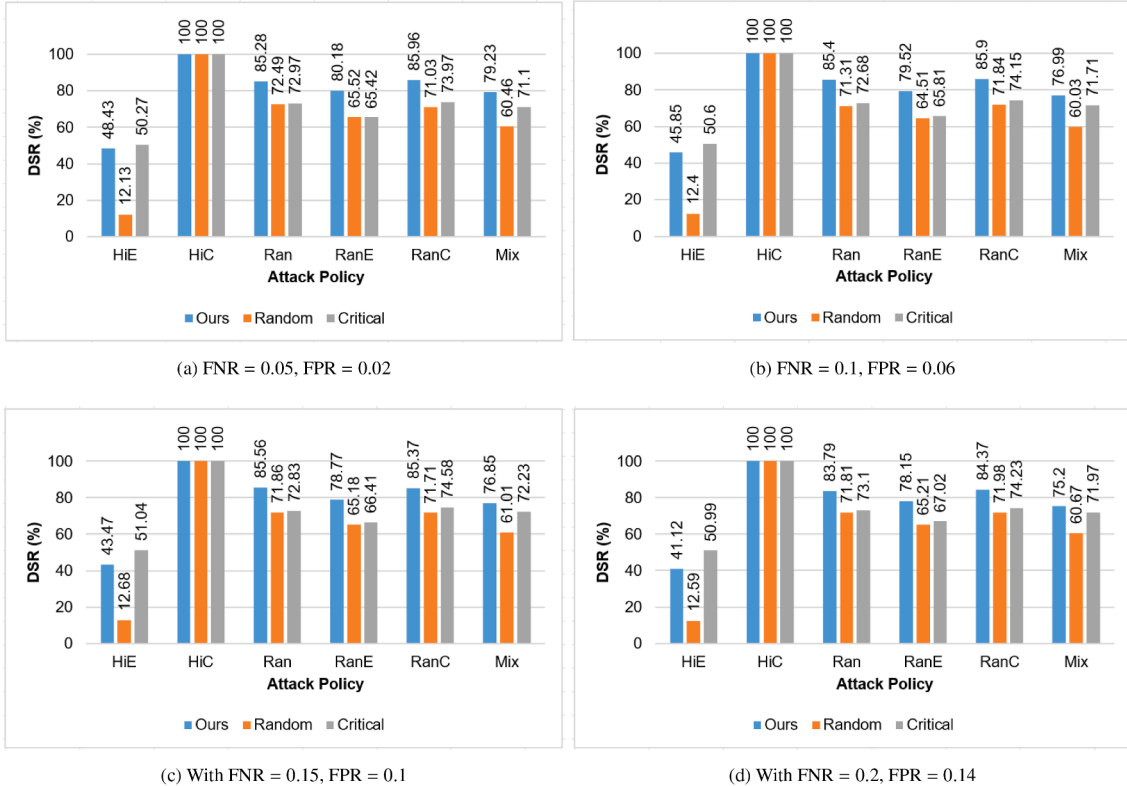


Fig. 10. Experimental results of defense strategies under diverse adversarial behaviors.

Table 3

DSP (%) across 10 experimental runs for each attack policy, with FNR = 0.05 and FPR = 0.02.

No. Trial	HiE	HiC	Ran	RanE	RanC	Mix
1	47.3	100	86.3	80.2	86.6	80.7
2	49.6	100	87.6	81.4	85.8	77.1
3	47.9	100	85.3	79.7	83.8	79.7
4	47.6	100	85.8	81.6	86.1	78.5
5	48.7	100	82.8	78.2	86.4	79.1
6	49.8	100	84.5	81.7	87.1	81.7
7	47.8	100	87.1	79.7	86	78.5
8	48.7	100	84.8	78.4	86.3	79.1
9	47.6	100	83.6	82.1	84.9	78.7
10	49.3	100	85	78.8	86.6	79.2
AVG	48.43	100	85.28	80.18	85.96	79.23

5.5.3. Scenario 2

The experimental results of Scenario 2 are presented in Fig. 11. The comparison between HMARL and MARL under identical conditions highlights several key findings. While both approaches achieve comparable FNR and FPR, HMARL consistently gains higher honeypot efficiency across all evaluated settings. For example, at FNR = 0.05, HMARL achieves a HiE of 48.43%, significantly surpassing RL with only 30.57%. Even under more adverse conditions (FNR = 0.20), HMARL maintains a HiE of 41.12%, whereas MARL drops sharply to 15.51%.

Beyond efficiency, HMARL demonstrates superior adaptability and resilience against diverse attack scenarios. In random attack environments, HMARL achieves 85.28% (Ran) and 80.18% (RanE), outperforming MARL with 77.76% and 69.48%, respectively. Similarly, in the mixed-attack scenario, HMARL reaches 79.23%, compared to 70.04% for MARL. These results confirm that the integration of a high-level strategy substantially enhances the system's robustness, resource utilization, and ability to sustain stable performance as adversarial conditions intensify.

The experimental evidence underscores that hierarchical architecture with high-level policy provides clear and consistent advantages over non-hierarchical architecture, ensuring more effective and resilient honeypot allocation in dynamic and uncertain environments.

5.5.4. Scenario 3

In Scenario 3, we evaluate the adaptability of the proposed model when the network environment undergoes significant changes, including modified access rules and updated vulnerability profiles, without retraining the RL component. The results, shown in Fig. 12, demonstrate several important findings.

First, when comparing the baseline method (SO) between the old and new environments, we observe a notable performance degradation. For instance, under the random attack (Ran) setting at FNR = 0.05, SO drops from 83.20% in the Old environment to 79.29% in the New one. Similarly, in the RanE case, the score decreases from 76.38% to 71.88%. This trend is consistent across different adversarial scenarios, highlighting the sensitivity of SO to environmental changes.

In contrast, our method demonstrates much stronger stability. In the same Ran case (FNR = 0.05), the performance remains nearly stable, with 85.28% in the Old environment and 83.34% in the New. In more complex cases, such as Mix, the performance drops only slightly from 79.23% to 77.54%, despite significant changes in topology and vulnerability distribution. This resilience is consistent across FNR/FPR levels, indicating that our model generalizes well to evolving network conditions without requiring retraining.

These results validate the robustness of the proposed approach in dynamic environments. While the baseline SO approach suffers noticeable drops in effectiveness under network changes, our method consistently sustains high performance. This confirms that our method achieves high effectiveness in its design, where the state encodes comprehensive information on compromised nodes, the network topology, and the vulnerabilities of individual hosts. Such a graph-based representation enables

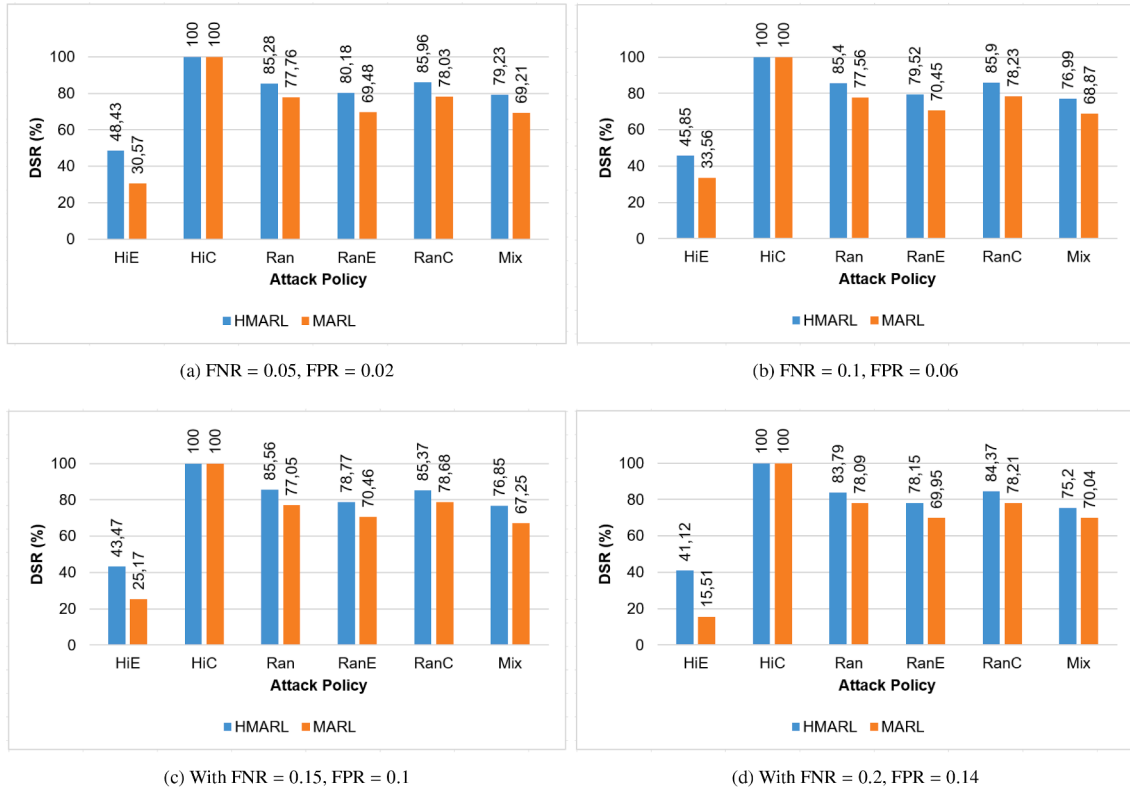


Fig. 11. Performance comparison between HMARL and MARL under different attack strategies.

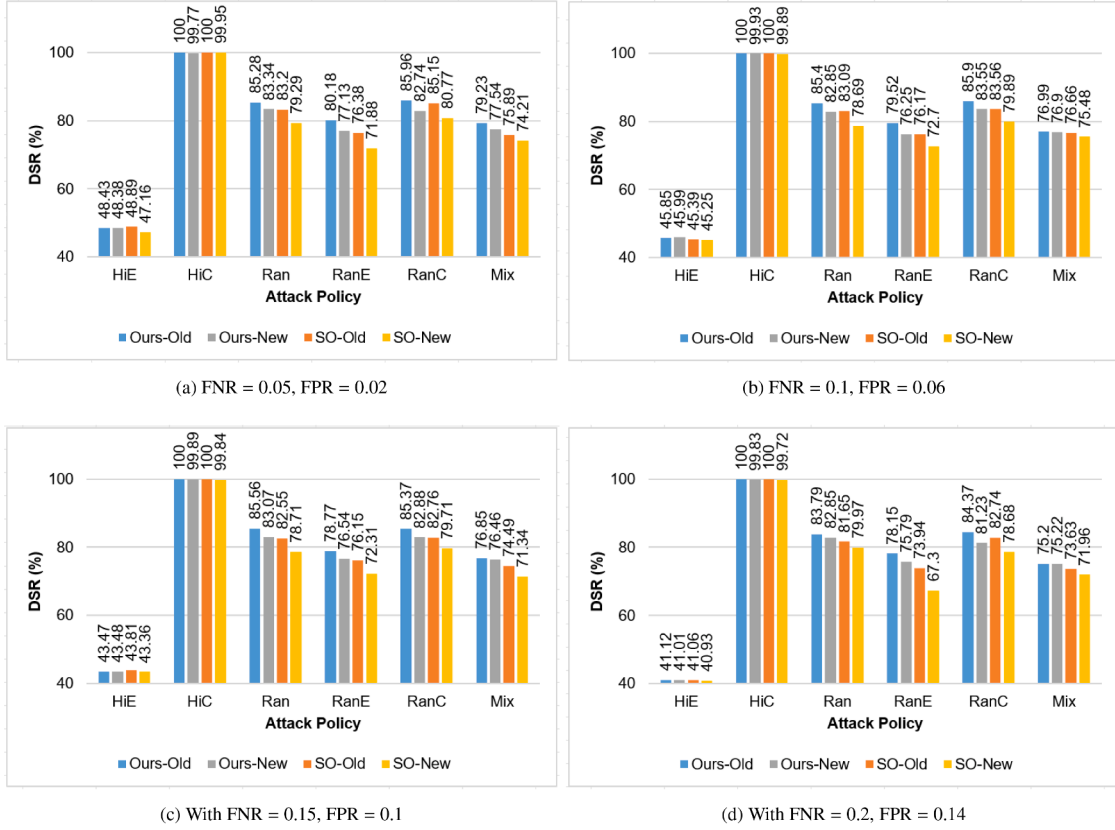


Fig. 12. Effectiveness of graph-based state representation.

the model to adapt to new environments more effectively than conventional approaches.

5.5.5. Scenario 4

The results shown in Fig. 13 demonstrate that the proposed method (*Ours*) consistently achieves the highest DSR across all attacker strategies and operating points. For deterministic attackers. Under stochastic and adaptive strategies (Ran, RanE, RanC, and Mix), which better approximate real-world adversaries, clear performance differences emerge. *Ours* maintains stable performance as FNR increases from 0.05 to 0.2, indicating strong robustness under increasing detection uncertainty.

Comparing *Ours* with *G16* highlights the impact of grouping granularity. While both methods employ HMARL with shared rewards, *G16* adopts finer-grained grouping, resulting in marginally lower performance in most cases. For example, under the Mix strategy at FNR = 0.2, *Ours* achieves 93.46% DSR compared to 91.88% for *G16*. This archive suggests that overly fine partitioning fragments the learning signal, reduces node observation, and increases coordination overhead, reducing the effectiveness of HMARL in large-scale networks.

The comparison with *NoG* clearly demonstrates the necessity of group-based honeypot allocation. Across all stochastic attack strategies, *NoG* exhibits a consistent performance drop of approximately 3–5%. For instance, at FNR = 0.1 under RanE, *Ours* achieves 83.28% DSR, whereas *NoG* drops to 76.38%. This gap indicates that grouping significantly enhances deception visibility and increases the likelihood of intercepting attackers beyond isolated, node-centric placements.

Finally, the comparison with *NoS* underscores the importance of reward sharing among agents. Although *NoS* retains group-based placement, disabling shared rewards leads to inferior performance, particularly under random and weighted-random attacks. At FNR = 0.15 under RanC, *Ours* achieves 81.98% DSR, compared to 79.93% for *NoS*. In the absence of reward sharing, agents tend to converge toward similar place-

ment choices, causing honeypots to be overly concentrated within a single group and reducing overall trapping coverage. By contrast, shared rewards in *Ours* promote coordinated exploration and balanced deployment, resulting in more effective and robust deception outcomes.

5.5.6. Scenario 5

The results of Scenario 5 highlight the feasibility of deploying the proposed defense framework in practice. As shown in Table 4 (all time values are measured from the start of an attack $t = 0$). On average, the system required 1.6 seconds to detect the intrusion, 1.99 seconds for the RL agents to compute the optimal honeypot placement, and 4.6 seconds to complete the deployment of deception nodes. This means that from the moment a node is compromised until honeypots are fully deployed, the entire process takes less than five seconds on average. The measured latency demonstrates that the proposed system can react quickly enough to disrupt adversarial activities before critical resources

Table 4

Time measurements for attack detection, decision, and honeypot deployment.

No. Trial	Attack time (t=0s)	Detection time (s)	Decision time (s)	Deployment time (s)
1	0	2.1	2.4	5.1
2	0	1.5	1.9	4.2
3	0	1.6	2.0	4.7
4	0	1.3	1.7	4.1
5	0	1.2	1.5	3.6
6	0	2.1	2.6	4.8
7	0	1.4	1.7	5.2
8	0	2.7	3.1	5.3
9	0	1.1	1.6	4.6
10	0	1.0	1.4	4.4
AVG	0	1.6	1.99	4.6
VAR	0	0.262	0.268	0.26

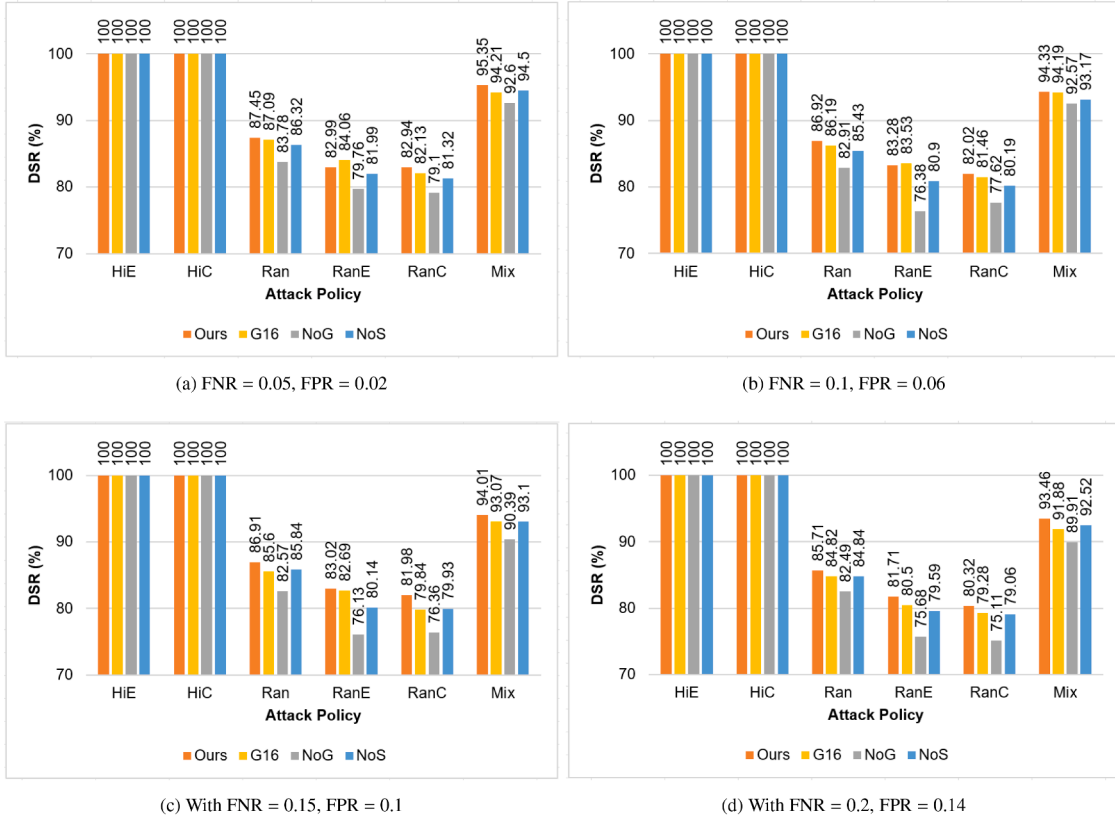


Fig. 13. The comparison of the proposed method and grouping and reward-sharing variants.

are reached. Moreover, the reported variance values (VAR) indicate that the system's performance is consistent across multiple trials, with low variability (~ 0.26) in detection, decision, and deployment times. Overall, these results confirm not only the effectiveness of the proposed approach but also its practicality for real-time deployment in operational environments, thereby complementing the effectiveness and adaptability already demonstrated in Scenarios 1–4.

5.6. Discussion

Compared with prior deception-placement strategies, our HMARL design offers three concrete advantages. First, by acting at the *group* level and sharing rewards across agents, the action space is decoupled from the number of honeypots, avoiding the combinatorial growth faced by single-agent or node-level policies [9].

Second, the architecture explicitly integrates uncertainty, both sensing errors (FPR/FNR) or attacker-policy variability, into training and evaluation, whereas many RL-based methods either assume perfect observability or a fixed adversary [10–12].

Third, the hierarchical split between high-level guidance and low-level allocation provides a structured decision-making process that enhances scalability and flexibility. Moreover, this hierarchical design increases the likelihood of successfully luring attackers into decoy environments, thereby strengthening the overall deception effectiveness.

Unlike game-theoretic approaches that rely on strong prior assumptions and simplified system dynamics to derive equilibrium solutions [8], our method prioritizes empirical robustness. This allows the framework to remain effective even when the underlying attack model is partially misspecified or uncertain.

To illustrate how this framework can be applied in practice, the proposed system can be deployed through a structured workflow. Network topology and routing data are collected to construct a network threat graph, after which hosts are grouped by function and risk to enhance

scalability. Honeypots are configured based on vulnerability scans and fine-tuned reinforcement learning models trained in both simulated and real environments. Finally, an orchestration module integrates with network devices or controllers to dynamically redirect malicious traffic while maintaining network safety and control.

While the proposed framework demonstrates promising results in scalability, adaptivity, and attack engagement, several limitations remain that warrant further investigation. Our environment encodes topology and exploitability via fixed-size matrices (e.g., adjacency/connection and vulnerability tensors). While simple and fast, matrix encodings lack the relational inductive bias of Graph Neural Networks (GNNs). They are less permutation-invariant, struggle to capture higher-order structures, and require padding or reindexing as the topology changes. GNN models could learn node/edge embeddings that better reflect lateral-movement dependencies and generalize across networks of different sizes.

Moreover, the current reward and state do not weight subnetworks by exposure/criticality (e.g., DMZ vs. internal segments). In practice, zones face different baseline compromise probabilities and business impacts; ignoring this heterogeneity can lead to suboptimal decoy budgets (over-protecting low-risk areas while under-protecting high-exposure or high-impact ones). Incorporating zone-level priors or learned risk scores into the allocation policy and reward shaping would align placement with operational risk. The network threat graph used in Hawkeyes primarily encodes static and structural features. While this abstraction enables reasoning over attack surfaces and defense decisions, it does not yet explicitly capture temporal or sequential patterns of attacker behavior such as lateral movement chains or exploit sequences. These dynamic patterns evolve over time and can reveal the progression of an intrusion. Incorporating such temporal dependencies into the enriched graph, for example, by tracking the time since compromise or using temporal edges that represent observed attack transitions, could further enhance the framework's stage-awareness and decision precision.

In addition, while including deterministic and randomized strategies (Ran, RanE, RanC, Mix), our current attacker model does not yet incorporate deception-aware or avoidance behaviors such as decoy probing, honeypot-averse routing, or low-and-slow movement. This simplification allows controlled comparison among defense strategies but may underestimate adaptive adversaries capable of recognizing and bypassing honeypots. Under such conditions, the role of honeypots shifts from pure trapping mechanisms to instruments for shaping attack paths, increasing attacker uncertainty, traversal cost, and exposure time within monitored regions. Accordingly, the objective of HMARL would evolve from maximizing capture probability to optimizing deception coverage and attack cost inflation. Future extensions should examine whether HMARL's advantage persists under such adaptive threat models, providing stronger robustness tests in realistic deception scenarios.

6. Related work

The problem of honeypot deployment has been extensively explored, with early work emphasizing static placement strategies. Although simple, these methods are often ineffective in dynamic environments or against sophisticated adversaries. To overcome such limitations, subsequent research has focused on more adaptive and intelligent deployment mechanisms.

One group of studies in this area primarily focused on modeling the attacker-defender interaction through static or semi-static frameworks. For example, game-theoretic models have been employed to determine the optimal location of the honeypot under adversarial behavior, considering resource limitations and network constraints. A notable example is the work in [7], where a dynamic game model accounts for network mobility and attacker preferences in tactical settings, introducing a scalable yet structured honeypot allocation method. Their method considers both attacker behavior and network connectivity, with the aim of reaching Nash equilibrium strategies for the placement of the honeypot under mobility constraints. Similarly, Bayesian-based approaches such as [8] integrate Stackelberg games with attack graphs to strategically guide honeypoint deployment and proactively anticipate adversary movements over time.

Parallel to game-theoretic modeling, another research direction uses machine learning techniques, particularly reinforcement learning, to dynamically adapt the deployment strategy according to changing network states. Building on this line, Anwar et al. [12] proposed a two-phase deception framework that integrates reinforcement learning with game theory under uncertainty. The proactive phase uses a zero-sum game model to optimize initial honeypot placement, while the reactive phase casts the problem as a POMDP solved with Monte-Carlo tree search. This enables defenders to operate effectively with partial observability of attacker actions, offering improved scalability and resilience compared to static, greedy, or random allocation strategies. Wang et al. [9] developed a Q-Learning-based deployment policy that adjusts the placement of deception resources according to the dynamics of the threat extracted from a threat penetration graph (TPG). Their work highlights the value of integrating state-based learning to improve honeypot coverage in high-risk zones. Radoglou-Grammatikis et al. [10] further extended this concept to ultra-dense networks beyond 5G. They modeled honeypot deployment as a multi-armed bandit problem and applied both Greedy and Q-learning methods to identify the optimal number of honeypots. This strategy enables scalable deployment decisions in wireless edge environments, which are prone to dynamic and heterogeneous threats. Meanwhile, Li et al. [11] proposed a deep RL-based deception strategy specifically for the cloud. Their method generates optimal decoy configurations across virtual machines, optimizing for asset concealment and adversary cost. These RL-based approaches represent an important advancement in adaptability, allowing honeypot sys-

tems to respond dynamically to attacker behavior and shifting network states.

In contrast, some other studies emphasize the realism and context awareness of honeypot systems, focusing on how the deception elements interact with the adversary in practice. Liu et al. [13] developed PHCG, a honeypot system specifically designed for industrial IoT (IIoT), especially programmable logic controllers (PLC). PHCG simulates authentic PLC communication by training a response model on real session data, thus deceiving the attacker without exposing critical assets. The system is lightweight and effective against insider threats that have penetrated the network defense system. Jafarian and Niakanlahiji [14] also went beyond traditional honeypot design by proposing MultiRHM, a framework that combines agility and deception in the network. MultiRHM anonymizes host identifiers, such as IP, MAC, DNS responses, and fingerprints, over time and across the network, targeting multistage intrusion campaigns based on reconnaissance and lateral movement. Both studies mark a shift toward multi-layered deception, focusing not only on strategic location but also on how honeypots operate and are perceived by attackers.

In another study, Wang et al. [15] introduced the ThreatInsight system, which further extends the role of honeypots beyond deception and into the realm of threat intelligence. Unlike approaches that focus solely on trap deployment or behavioral mimicry, ThreatInsight uses IP data derived from honeypoints as input for knowledge graph-based reasoning. Combining deception elements with threat intelligence enables early-stage threat detection, attacker attribution, and situational awareness. This integration represents a meaningful step toward predictive defense, as it leverages both attacker interactions and contextual threat analysis. Although the system demonstrates the potential to connect honeypot deployments to higher-level intelligence operations, relying on the quality and timeliness of external threat data remains a practical limitation. ThreatInsight, however, illustrates the expanding role of honeypots, not just passive traps but as active sensors that contribute to the broader cyber defense ecosystem.

Beyond honeypot allocation, recent advances in machine learning emphasize knowledge aggregation and cross-model representation learning to enhance decision-making under complex, multi-source environments. For instance, the HMKD framework [31] introduces heterogeneous mutual knowledge distillation for wearable human activity recognition (HAR), enabling bidirectional knowledge transfer between teacher and student models with different architectures. It achieves notable accuracy improvements across multiple HAR datasets. Additionally, the KATN model [32] proposes a transformer-based approach for multivariate time series classification that aggregates local and global features through modified residual and attention networks, outperforming existing transformer variants on 13 University of East Anglia (UEA) datasets.

Overall, current research reflects a clear shift from static honeypot setups to adaptive, intelligent, and context-aware systems. Approaches based on game theory, reinforcement learning, and behavioral modeling have significantly advanced the field. However, key challenges remain, including limited contextual modeling in RL-based methods [9,10], oversimplified network abstractions in game-theoretic models [7,8], and insufficient attention to practical deployment constraints such as cost, fidelity, and system integration. These open issues motivate our research focus on designing honeypot strategies that are not only adaptive but also practical and robust for real-world cyber defense scenarios. Table 5 summarizes representative related works. It highlights key aspects, including the methodology employed, the adaptability of each approach to different attack scenarios, whether active deception mechanisms are integrated, whether the approach explicitly considers uncertainty, such as incomplete knowledge about attacker behavior or system state, and the scalability of the proposed solutions for larger or more complex environments.

Table 5
Overview of related work.

Author	Methodology	Ability to Adapt	Active Deception	Detection noise(FPR/FNR)	Adversary Policy	Scalability
Wang et al. [9]	Q-Learning; Threat Penetration Graph (TPG)	High	Yes	Yes	Single	Medium
Li et al. [11]	Deep RL; Utility-driven decoy placement	High	Yes	No	Single	Low
Jafarian et al. [14]	MultiRHM	High	Yes	Partial	Not mentioned	Low
Sayed et al. [7]	Dynamic Game Theory; Mobility Modeling	High	Yes	No	Single	Low
Radoglou et al. [10]	RL (beyond 5G/ultra-dense)	Medium	Yes	No	Single	High
Wang et al. [15]	Threat Intelligence; Knowledge Graph	Medium	No	No	Not mentioned	High
Liu et al. [13]	PHCG (PLC simulation)	Medium	Yes	No	Not mentioned	High
Wang et al. [8]	Stackelberg Game; Bayesian Attack Graph	High	Yes	No	Multiple	Medium
Anwar et al. [12]	Game Theory; RL	High	Yes	No	Multiple	Medium
Ours	Hierarchical Multi-agent RL; Network Threat Graph	High	Yes	Yes	Multiple	High

7. Conclusion

This study introduced Hawkeyes, a hierarchical multi-agent reinforcement learning (HMARL) framework for adaptive honeypot allocation in cyber deception. By integrating stage-aware high-level strategies with fine-grained low-level placement decisions and leveraging action-space reduction through grouping and multi-agent decomposition, the framework addresses key challenges of scalability, coordination, and intrusion-stage awareness. The enriched graph-based state representation further enables context-rich decision-making by capturing compromised nodes, topology, and host vulnerabilities. Experimental evaluations demonstrated that Hawkeyes consistently outperforms baseline RL approaches, achieving higher trapping efficiency, superior stage-aware defense, and stable performance under diverse network conditions, all while ensuring real-time deployability. These results highlight the potential of HMARL to advance adaptive cyber defense strategies.

8. Future work

In future work, we will explore more graph-based representations (e.g., GNNs) to better capture relational structures and dynamic topologies beyond fixed-size matrices. In addition, incorporating zone-level exposure and criticality into the reward and state design could prevent resource misallocation across low and high-risk areas.

We also plan to extend the attack model to include deception-aware and avoidance behaviors (e.g., decoy probing, honeypot-averse routing, low-and-slow lateral movement). In these settings, attackers may partially identify or suspect honeypots, prompting the defender to adapt its strategy from baiting to strategic path manipulation and uncertainty amplification. This will require redefining the reward structure to capture deception effectiveness beyond binary trapping outcomes, such as increased attack latency, forced rerouting, and prolonged interaction with deceptive regions. Such adaptive adversaries enable us to evaluate how HMARL performs against more realistic, stealthy, and strategic attack patterns, thereby validating its robustness beyond vulnerability-driven or randomized attacker baselines.

A direction for future research is to extend the enriched graph representation with temporal and behavioral features that reflect real attacker dynamics. This could involve using time-stamped features to represent exploit sequences or lateral movement chains to capture evolving attack trajectories. Integrating these temporal characteristics would enable Hawkeyes to reason over both spatial and temporal aspects of cyber attacks, leading to more adaptive and predictive deception strategies.

In addition, we aim to investigate the transferability of learned policies across different network environments (e.g., from testbed to production) using fine-tuning and incremental learning to minimize retraining costs. Another potential extension is to generalize the hierarchical framework to orchestrate diverse deception assets, such as decoy documents or fake credentials, under the same multi-level control scheme, moving toward a unified, adaptive cyber deception ecosystem.

Another promising direction is to evaluate the robustness of the framework under adversarial conditions. Hawkeyes can be deployed

in environments with adversarial perturbations to assess how well its policies withstand manipulation and deception. Furthermore, incorporating adversarial training could enhance policy resilience and ensure stable decision-making even when the environment provides deceptive or noisy feedback.

CRedit authorship contribution statement

Hien Do Hoang: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Conceptualization; **Trong-Nghia To:** Writing – review & editing, Writing – original draft, Software, Methodology, Conceptualization; **Ngo Duc Hoang Son:** Writing – review & editing, Writing – original draft; **Khoa Ngo-Khanh:** Writing – review & editing, Writing – original draft, Software; **Nguyen Tan Cam:** Writing – review & editing, Writing – original draft, Validation; **Van-Hau Pham:** Writing – review & editing, Writing – original draft, Validation, Methodology, Conceptualization.

Data availability

No data was used for the research described in the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This research is funded by Vietnam National University HoChiMinh City (VNU-HCM) under grant number NCM2025-26-01.

References

- [1] L. Zhang, V.L.L. Thing, Three decades of deception techniques in active cyber defense - Retrospect and outlook, *Comput. Secur.* 106 (2021) 102288.
- [2] A. Javadpour, F. Ja'fari, T. Taleb, M. Shojafar, C. Benzaid, A comprehensive survey on cyber deception techniques to improve honeypot performance, *Comput. Secur.* 140 (2024) 103792.
- [3] R. Colbaugh, K. Glass, Proactive defense for evolving cyber threats, in: *Proceedings of 2011 Del InsThinspace IEEE International Conference on Intelligence and Security Informatics*, 2011, pp. 125–130.
- [4] M. Zhu, A.H. Anwar, Z. Wan, J.-H. Cho, C.A. Kamhoua, M.P. Singh, A survey of defensive deception: approaches using game theory and machine learning, *IEEE Commun. Surv. Tutor.* 23 (4) (2021) 2460–2493.
- [5] G. Ikuomenisan, Y. Morgan, Meta-Review of recent and landmark honeypot research and surveys, *J. Inf. Secur.* 13 (4) (2022).
- [6] Z. Morić, V. Dakić, D. Regvart, Advancing cybersecurity with honeypots and deception strategies, *Informatics* 12 (1) (2025).
- [7] M.A. Sayed, A.H. Anwar, C. Kiekintveld, C. Kamhoua, Honeypot allocation for cyber deception in dynamic tactical networks: a game theoretic approach, in: *International Conference on Decision and Game Theory for Security*, 2023, pp. 195–214.
- [8] R. Wang, C. Yang, X. Deng, Y. Zhou, Y. Liu, Z. Tian, Turn the tables: proactive deception defense decision-making based on bayesian attack graphs and stackelberg games, *Neurocomputing* 638 (2025) 130139.

- [9] S. Wang, Q. Pei, J. Wang, G. Tang, Y. Zhang, X. Liu, An intelligent deployment policy for deception resources based on reinforcement learning, *IEEE Access* 8 (2020) 35792–35804.
- [10] P. Radoglou-Grammatikis, P. Sarigiannidis, P. Diamantoulakis, T. Lagkas, T. Saoulidis, E. Fountoukidis, G. Karagiannidis, Strategic honeypot deployment in ultra-dense beyond 5G networks: a reinforcement learning approach, *IEEE Trans. Emerg. Top. Comput.* (2022).
- [11] L.I. Huanruo, G. Yunfei, H. Shumin, et al., Defensive deception framework against reconnaissance attacks in the cloud with deep reinforcement learning, *Sci. China Inf. Sci.* 65 (7) (2022) 170305.
- [12] A.H. Anwar, C.A. Kamhoua, N.O. Leslie, C. Kiekintveld, Honeypot allocation for cyber deception under uncertainty, *IEEE Trans. Netw. Serv. Manage.* 19 (3) (2022) 3438–3452.
- [13] H. Liu, Y. Zhou, B. Fang, Y. Sun, N. Hu, Z. Tian, PHCG: PLC Honeypoint communication generator for industrial IoT, *IEEE Trans. Mob. Comput.* (2024).
- [14] J.H. Jafarian, A. Niakanlahiji, MultiRHM: defeating multi-staged enterprise intrusion attacks through multi-dimensional and multi-parameter host identity anonymization, *Comput. Secur.* 124 (2023) 102958.
- [15] Z. Wang, Y. Zhou, H. Liu, J. Qiu, B. Fang, Z. Tian, Threatinsight: innovating early threat detection through threat-Intelligence-Driven analysis and attribution, *IEEE Trans. Knowl. Data Eng.* (2024).
- [16] P. Beltrán-López, M.G. Pérez, P. Nespole, Cyber deception: taxonomy, state of the art, frameworks, trends, and open challenges, *IEEE Commun. Surv. Tutor.* (2025).
- [17] M. Al-Emran, Hierarchical reinforcement learning: a survey, *Int. J. Comput. Digit. Syst.* 4 (02) (2015).
- [18] X. Wang, S. Wang, X. Liang, D. Zhao, J. Huang, X. Xu, B. Dai, Q. Miao, Deep reinforcement learning: a survey, *IEEE Trans. Neural Netw. Learn. Syst.* 35 (4) (2024) 5064–5078.
- [19] A.G. Barto, S. Mahadevan, Recent advances in hierarchical reinforcement learning, *Discrete Event Dyn. Syst.* 13 (1) (2003) 341–379.
- [20] O. Nachum, S. Gu, H. Lee, S. Levine, Data-efficient hierarchical reinforcement learning, in: *Advances in Neural Information Processing Systems*, 2018.
- [21] T.G. Dietterich, Hierarchical reinforcement learning with the MAXQ value function decomposition, in: *Journal of Artificial Intelligence Research*, 13, 2000, pp. 227–303.
- [22] S. Gronauer, K. Diepold, Multi-agent deep reinforcement learning: a survey, *Artif. Intell. Rev.* 55 (2) (2022) 895–943.
- [23] L. Busoniu, R. Babuska, B. De Schutter, A comprehensive survey of multiagent reinforcement learning, *IEEE Trans. Syst. Man Cybernet. Part C (Appl. Rev.)* 38 (2) (2008) 156–172.
- [24] Z. Ning, L. Xie, A survey on multi-agent reinforcement learning and its application, *J. Automat. Intell.* 3 (2) (2024) 73–91.
- [25] J. Foerster, G. Farquhar, T. Afouras, N. Nardelli, S. Whiteson, Counterfactual multi-agent policy gradients, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, 32, 2018.
- [26] R. Lowe, Y. Wu, A. Tamar, J. Harb, A. Pieter, I. Mordatch, Multi-agent actor-critic for mixed cooperative-competitive environments, in: *Advances in Neural Information Processing Systems*, 2017, pp. 6382–6393.
- [27] S. Security, TrapX DeceptionGrid - deception without Limits, 2019, (<https://promo.softprom.com/trapx>).
- [28] SentinelOne, Deception for Inside-the-Network Threat Detection, 2023, (<https://assets.sentinelone.com/deception/deception-for-inside-protect/penalty-\@M-the-network-threat-detection-en>). White Paper.
- [29] J. Foerster, I.A. Assael, N. de Freitas, S. Whiteson, Learning to communicate with deep multi-Agent reinforcement learning, in: *Advances in Neural Information Processing Systems*, 29, 2016.
- [30] T. Rashid, M. Samvelyan, C.S. de Witt, G. Farquhar, J. Foerster, S. Whiteson, Monotonic value function factorisation for deep multi-Agent reinforcement learning, *J. Mach. Learn. Res.* 21 (178) (2020) 1–51.
- [31] Z. Xiao, H. Xing, R. Qu, H. Li, X. Cheng, L. Xu, L. Feng, Q. Wan, Heterogeneous mutual knowledge distillation for wearable human activity recognition, *IEEE Trans. Neural Netw. Learn. Syst.* 36 (9) (2025) 16589–16603.
- [32] Z. Xiao, H. Xing, R. Qu, H. Li, H. Tong, S. Luo, J. Song, L. Feng, Q. Wan, Knowledge aggregation transformer network for multivariate time series classification, *IEEE Trans. Big Data* (2025) 1–17.



Hien Do Hoang received a B.Sc. degree in Computer Networks and Data Communications in 2017 from the University of Information Technology, Vietnam National University Ho Chi Minh City (UIT, VNU-HCM). From 2017 to 2018, the author worked in the industry in the field of networking and information security. In 2020, he earned an M.Sc. degree in Information Technology from UIT, VNU-HCM. Since 2023, he has been a Ph.D. candidate at UIT, VNU-HCM. Currently, he is a member of the Information Security Laboratory (InSecLab) at UIT, VNU-HCM. His research interests include network security and administration, cyber deception, moving target defense, virtualization and cloud computing, and web security.



Trong-Nghia To received the B.Eng. degree in Information Security from the University of Information Technology, Vietnam National University Ho Chi Minh City (UIT-VNU-HCM) in 2022, and the M.Eng. degree in Information Security in 2024. From February 2022 to the present, he has been working as a researcher at the Information Security Laboratory (InSecLab), UIT. His research interests include Malware Analysis, Software Security, Machine Learning, and Explainable Artificial Intelligence.



Ngo Duc Hoang Son is pursuing the Master of Science degree in Information Security from the University of Information Technology (UIT) Vietnam National University Ho Chi Minh City (VNU-HCM). He also works as a researcher in Information Security Laboratory (InSecLab) UITVNU- HCM. His main research interests include Software Defined Networking, Malware Analysis and Machine Learning-based Cybersecurity.



Khoa Ngo-Khanh received the B.Eng. degree in Information Security from the University of Information Technology, Vietnam National University Ho Chi Minh City (UIT-VNU-HCM) in 2018. He is pursuing an M.Sc. degree in Information Technology at UIT-VNU-HCM. From 2022 until now, he works as a member of a research group at the Information Security Laboratory (InSecLab) at UIT. His main research interests are Information security, including Software Vulnerability, Fuzzing, Web Security.



Dr. Nguyen Tan Cam obtained his bachelor's degree in Information Technology from the University of Science, Vietnam National University Ho Chi Minh City in 2006. He received his master's degree in Information Systems from this university in 2010. He earned his Ph.D. in Information Technology from the University of Information Technology, Vietnam National University Ho Chi Minh City (UIT VNU-HCM). He is a lecturer at UIT VNU-HCM. His main research interests include mobile security, network security, machine learning, and deep learning.



Van-Hau Pham obtained his Bachelor degree in Computer Science from the University of Natural Sciences of Hochiminh City in 1998. He pursued his Master degree in Computer Science from the Institut de la Francophonie pour l'Informatique (IFI) in Vietnam from 2002 to 2004. Then he did his internship and worked as a full-time research engineer in France for 2 years. He then persuaded his PhD thesis on network security under the direction of Professor Marc Dacier from 2005 to 2009. He is now a lecturer at the University of Information Technology, Vietnam National University Hochiminh City (VNU-HCM), Hochiminh City, Vietnam. His main research interests include network security, system security, mobile security, and cloud computing.